树/Trees

11.1 树的概念/Introduction of Trees

11.2 树的应用/Applications of Trees

11.3 树的遍历/Tree Traversal

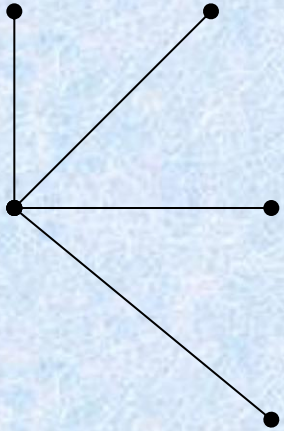
11.4 生成树/Spanning Trees

11.5 最小生成树/ minimum Spanning Trees

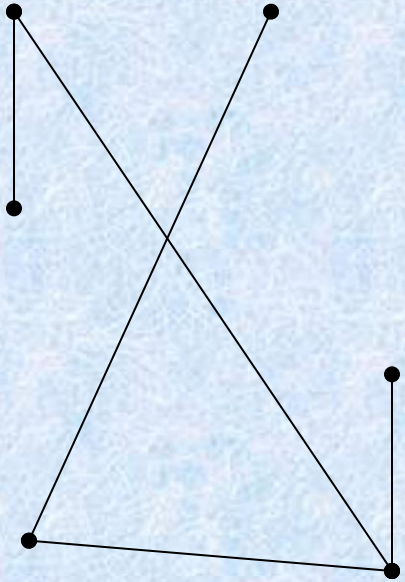
§ 11.1: Introduction to Trees

- A *tree* is a connected undirected graph with no simple circuits.
  - **Theorem:** There is a unique simple path between any two of its nodes.
- An undirected graph without simple circuits is called a *forest*.
  - You can think of it as a set of trees having disjoint sets of nodes.

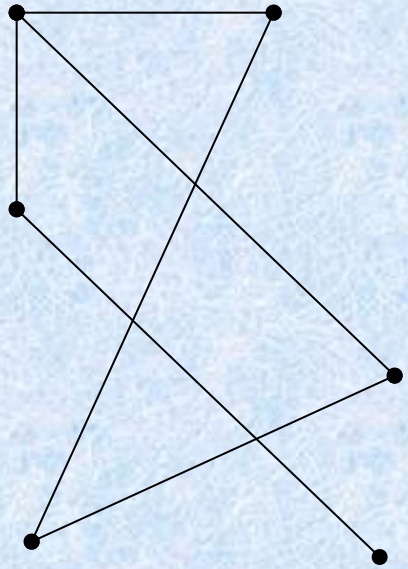
Examples of Trees and Graphs that are not Trees



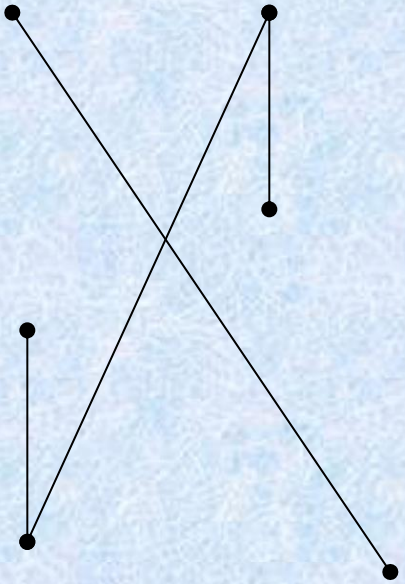
G1



G2



G3



G4

给定图T,以下关于树的定义是等价的:

1) 无回路的连通图;

2) 无回路且 $e = v - 1;$

3) 连通且 $e = v - 1;$

4) 无回路,但增加一条新边,得一个且仅一个回路;

5) 连通但删去后便不连通;

6) 每一对结点间有一条且仅有一条路.

§1　树的概念

[定义]树： 一个无简单回路的连通无向图称为一棵树/Tree。记为T。

[定义]林： 无简单回路的无向图是林/Forests。

[定理1]树的基本性质：

（1）树是平面图。

(2）树中任二顶点间都有唯一道路。

(3) 去掉树的任一边，成了林。

(4)在树上任添一条边，产生唯一回路。

(5)设T＝（V，E），｜V｜＝v，｜E｜＝e，则v＝e＋1。

树是平面图，且无回路，区域数γ＝1，满足欧拉公式v－e＋γ＝2，即v＝e＋1。

也可用强归纳法证：（对V归纳）

（1）v≤2，显然成立

（2）假设v＜p时成立，v＝p时，先移去一条边，成了两棵树T_1和T_2，有$v_1=e_1+1$，$v_2=e_2+1$，且$e_1+e_2=e-1$，故$v=v_1+v_2=e_1+1+e_2+1=(e_1+e_2+1)+1=e+1$。

Rooted Trees

- A *rooted tree* is a tree in which one node has been designated the *root*.
  - Every edge is (implicitly or explicitly) directed away from the root.
- You should know the following terms about rooted trees:
  - Parent, child, siblings, ancestors, descendents, leaf, internal node, subtree.

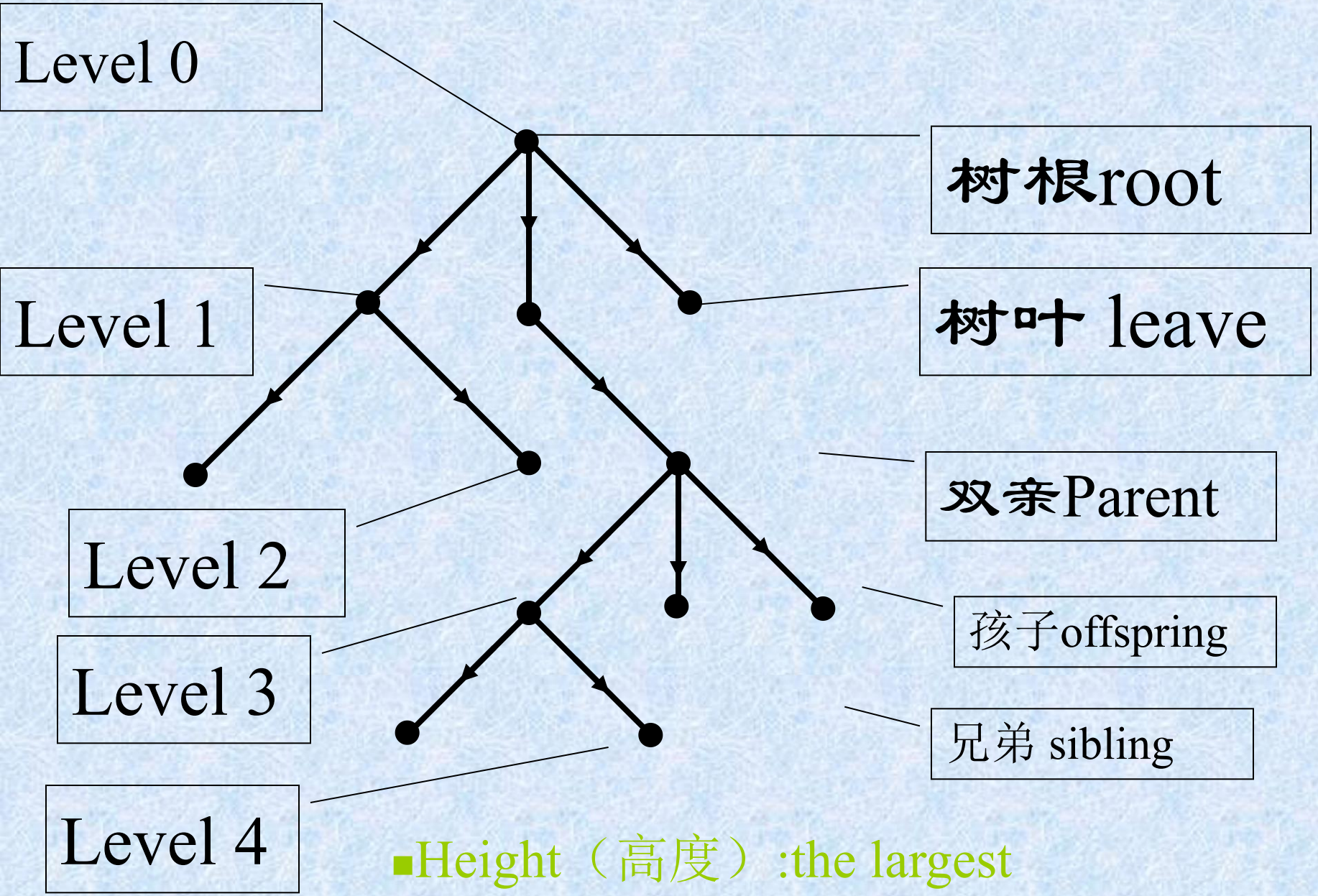
[定义]有向树：如果有向图Ｇ＝（Ｖ，Ｅ）对应的无向图是树，称Ｇ为有向树/directed tree。

[定义]有根树：如果有向树存在唯一一个入次为 0 的顶点，其余顶点入次均为１，称此有向树为有根树/rooted tree。

[定义] 根：有根树中入次为 0 的顶点称为根/root。

[定义] 叶：有根树中出次为 0 的顶点称为叶/leaf。

[定义]枝点：有根树中除叶以外的顶点均为枝点/internal vertices。



- Height（高度）:the largest level number of the tree

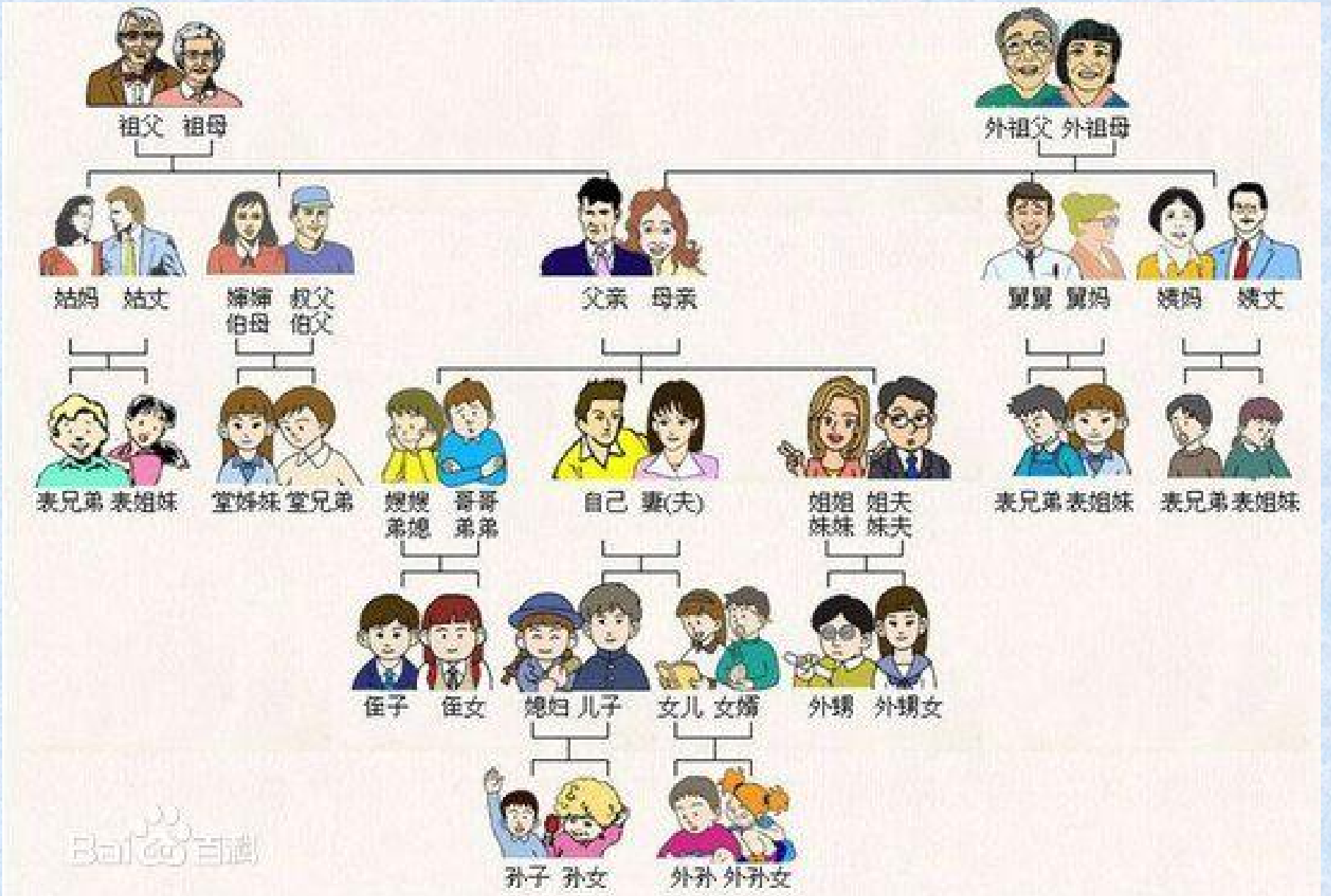
[定义] 父亲，儿子，兄弟：

有根树G＝（V,E），若（a，b）∈E，则称a是b的父亲/parent，b是a的儿子/child。

若（a，b_1）∈E，（a，b_2）∈E，称b_1与b_2互为兄弟/siblings。

[定义] 祖先，后裔：

有根树中，若a到c有道路存在，则称a是c的祖先/ancestors，c是a的后裔descendants。


祖父 祖母
外祖父 外祖母
姑妈 姑丈
婶婶 叔父
伯母 伯父
父亲 母亲
舅舅 舅妈
姨妈 姨丈
表兄弟 表姐妹
堂姊妹 堂兄弟
嫂嫂 哥哥
弟媳 弟弟
自己 妻(夫)
姐姐 姐夫
妹妹 妹夫
表兄弟 表姐妹
表兄弟 表姐妹
侄子 侄女
媳妇 儿子
女儿 女婿
外甥 外甥女
孙子 孙女
外孙 外孙女

[定义]子树：有根树 T ＝（V，E），a 是 T 的枝点，V_a是 a 及其后裔的集合，E_a是 a 到各后裔道路上边的集合，则 T_a＝（V_a，E_a）称为 T 的以 a 为根的子树/subtree。

注：以 a 为根的子树,包含根顶点 a .
a 的子树:以 a 的儿子为根的子树。

[定义]有序树：对枝点的儿子规定一个次序，如长子，次子等，则此有根树称为有序树/ordered tree。

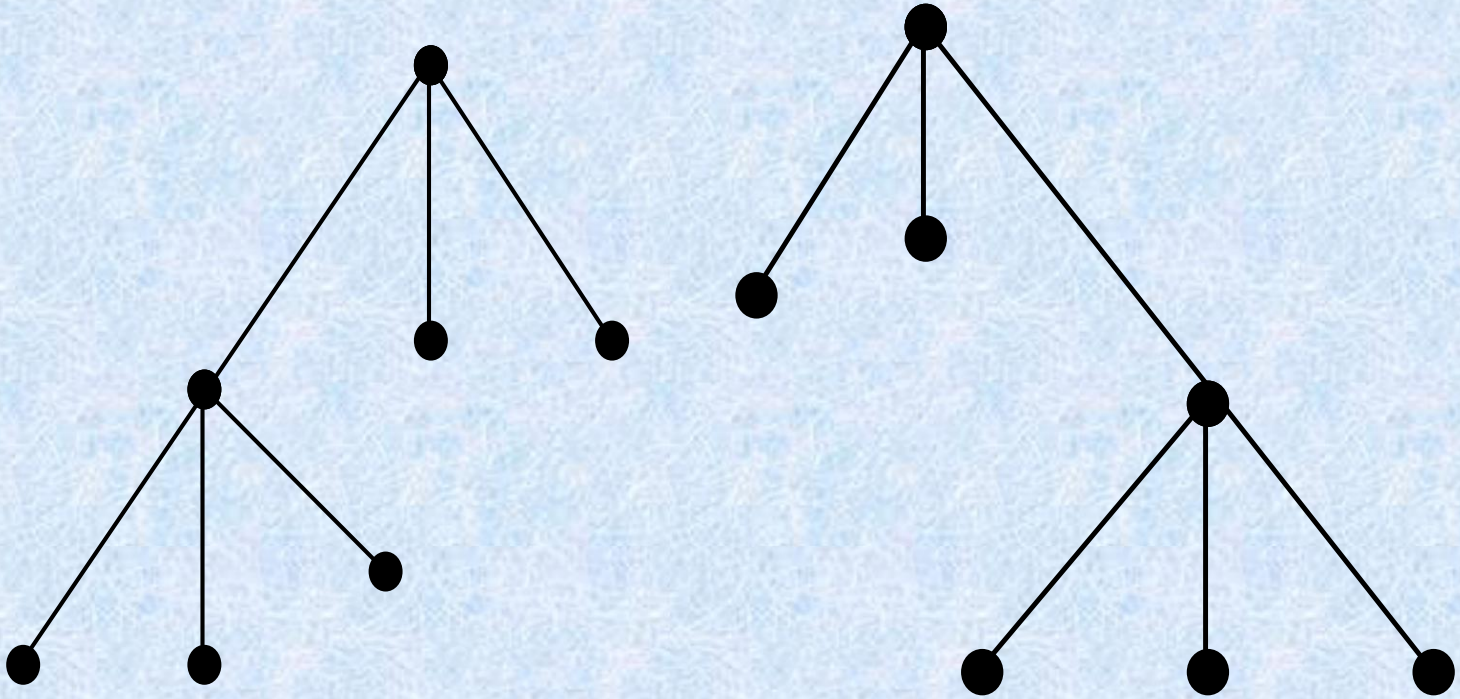
[定义]有序树中的

左儿子/left child：枝点的左边儿子
右儿子/right child ：枝点的右边儿子
左子树/left subtree ：左儿子为根的子树
右子树/right subtree ：右儿子为根的子树

如下图

作为有根树：同构

作为有序树：不同构



[定义]n元树：每个枝点最多有n个儿子的有根树称为n元树/n-ary tree。

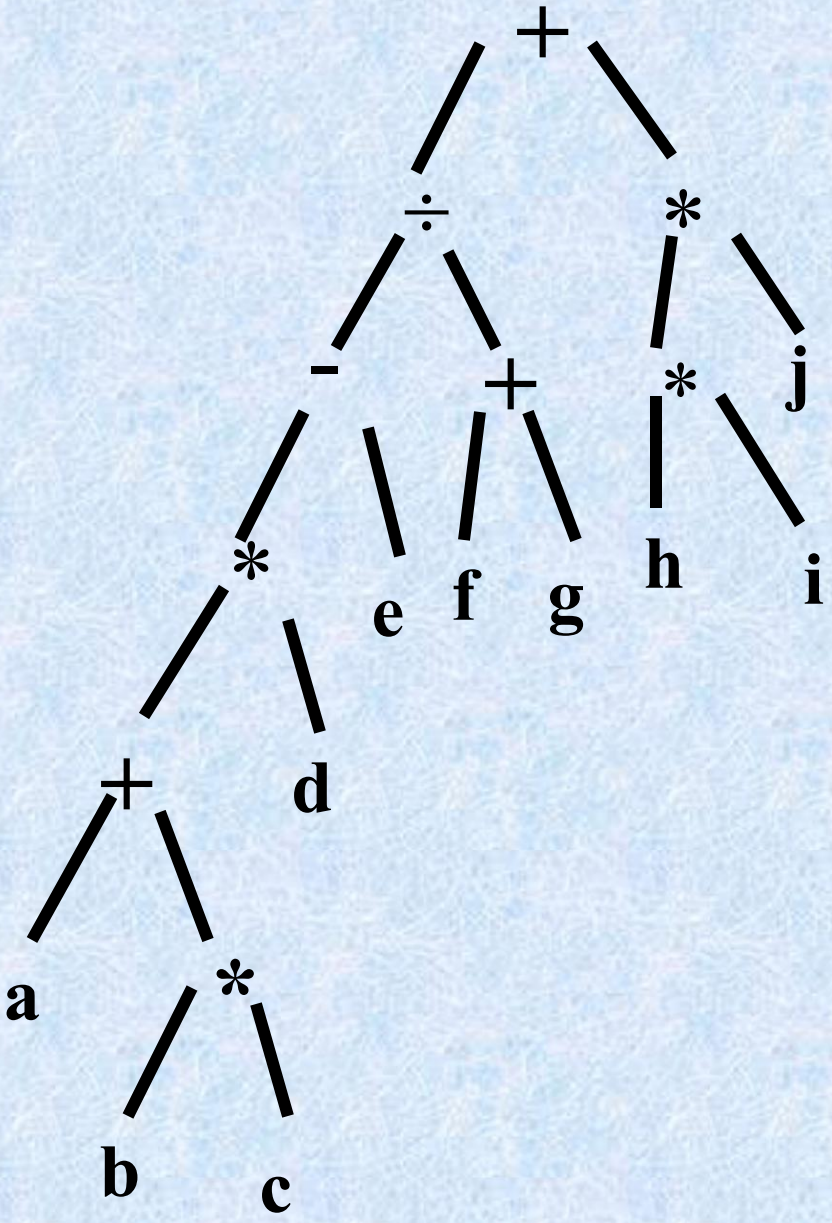
[定义]n元正规树：每个枝点恰有n个儿子的有根树称为n元正规树/full n-ary tree。

[定义]n元树n=2时称为二元树或二叉树/binary tree。

例:

算术表达式的树表示(二元树):

（（ａ＋ｂ×ｃ）×ｄ－ｅ）/（ｆ＋ｇ）＋ｈ×ｉ×ｊ


+
÷
*
-
+
*
j
*
e
f
g
h
i
+
d
a
*
b
c

n-ary trees

- A rooted tree is called *n*-ary if every internal vertex has no more than *n* children.
- It is *full* if every internal vertex has *exactly* *n* children.
- A 2-ary tree is called a *binary tree*.

n叉 树: 每个结点的出度小于或等于n的根树.

完全n叉树: 每个结点的出度恰好等于n或零的根树.

正则n叉树: 所有树叶层次相同的完全n叉树.

[定义]顶点道路长度和层：顶点 a 的道路长或顶点的层/level是指从根到 a 的边的条数，记为l（ a ）。

[定义]树高：有根树的树高/height指到根最远的顶点 a 的道路长度。

即 h＝max[l(a)]　a∈V

性质：n元树高度为 h ，最多能有n^h片叶。

[定义]n元完全树：如果树高为 h 的n元树有n^h片叶，称此树为n元完全树/complete n-ary tree。

注： 完全树当然是正规的。

Ordered Rooted Tree

- A rooted tree where the children of each internal node are ordered.
- In ordered binary trees, we can define:
  - left child, right child
  - left subtree, right subtree
- For *n*-ary trees with *n*>2, can use terms like “leftmost”, “rightmost,” etc.

Trees as Models

- Can use trees to model the following:
  - Saturated hydrocarbons
  - Organizational structures
  - Computer file systems
- In each case, would you use a rooted or a non-rooted tree?

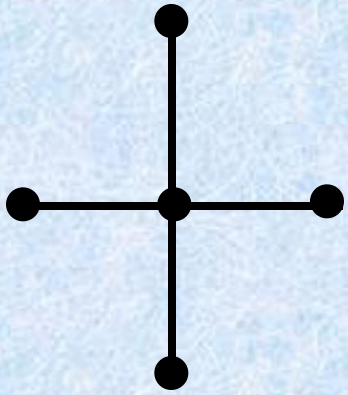
树:连通且无回路的无向图.

树叶:树中度数为1的结点.

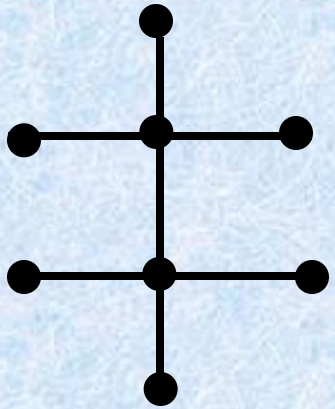
分枝或内点:度数大于1的结点.

森林:每个连通分支是树的无向图.

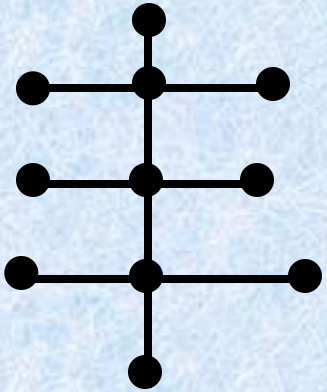
化学同分异构物



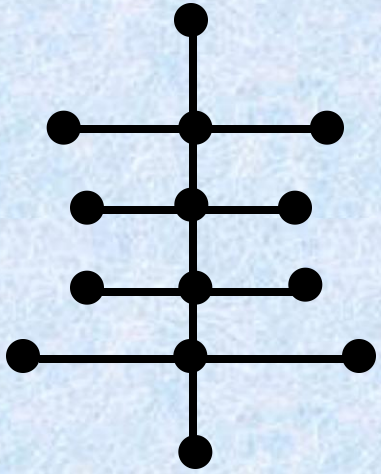
甲烷



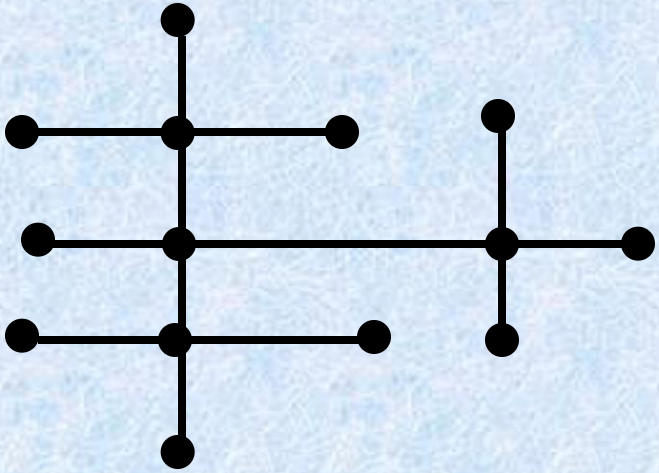
乙烷



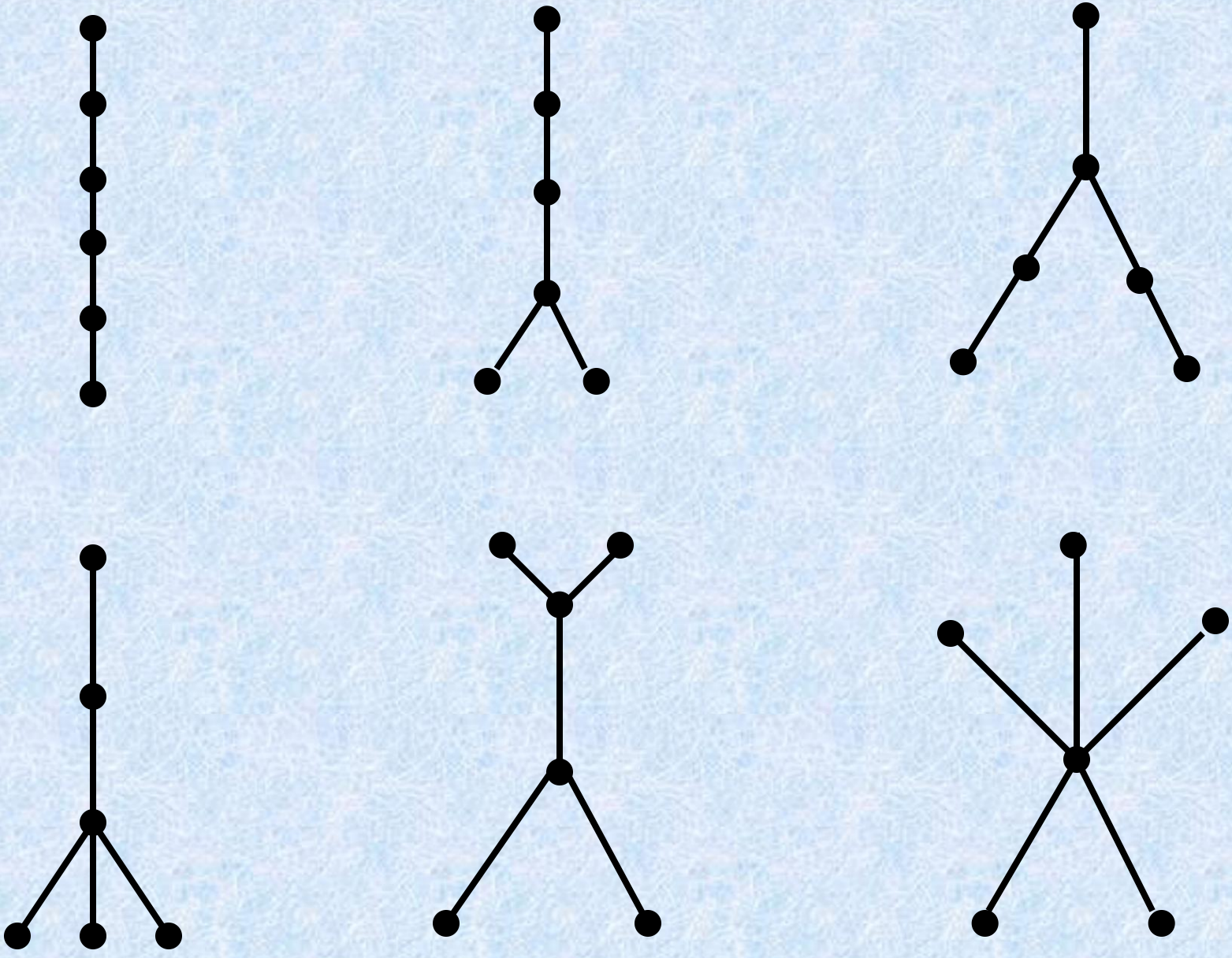
丙烷



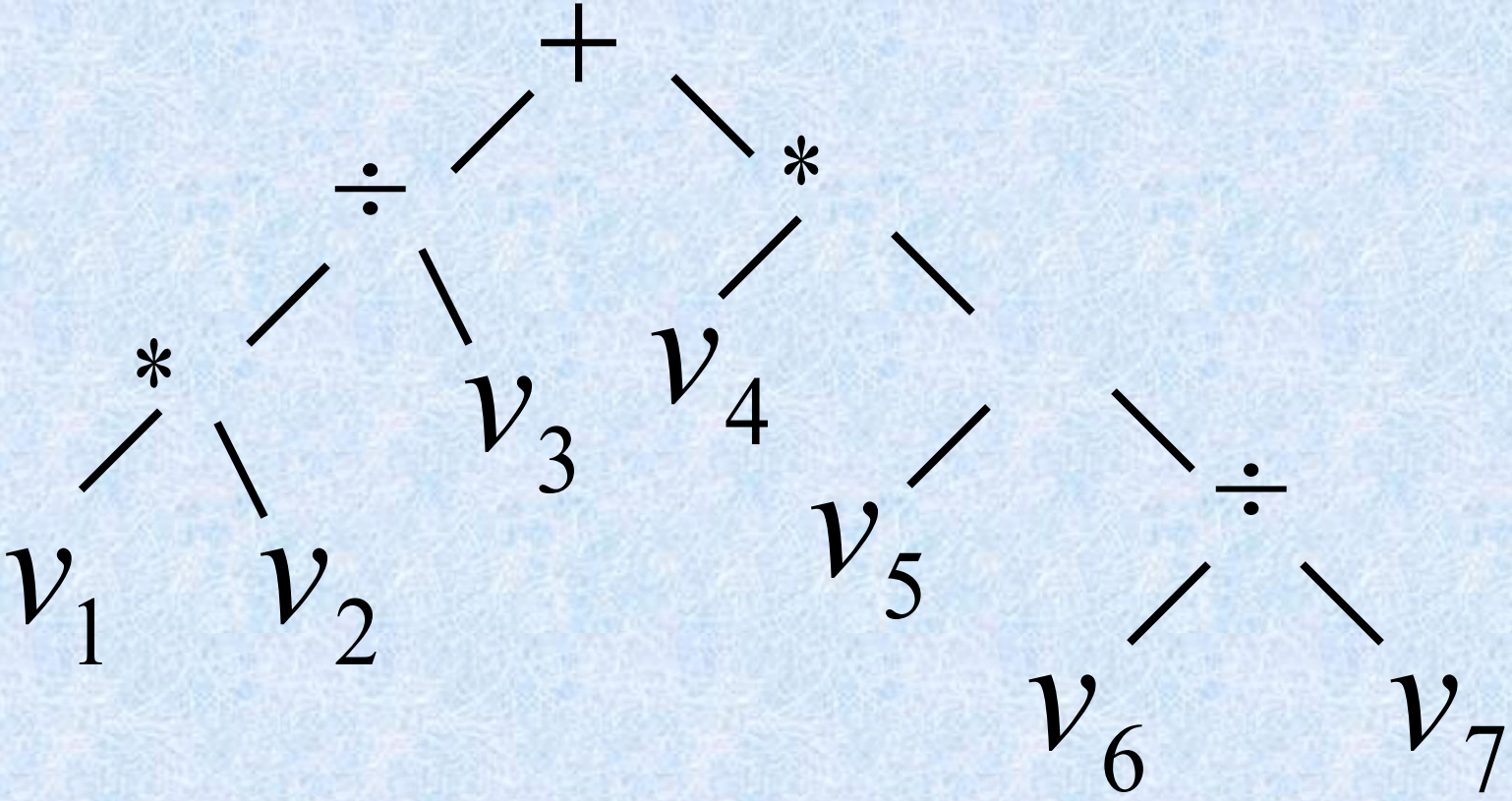
丁烷



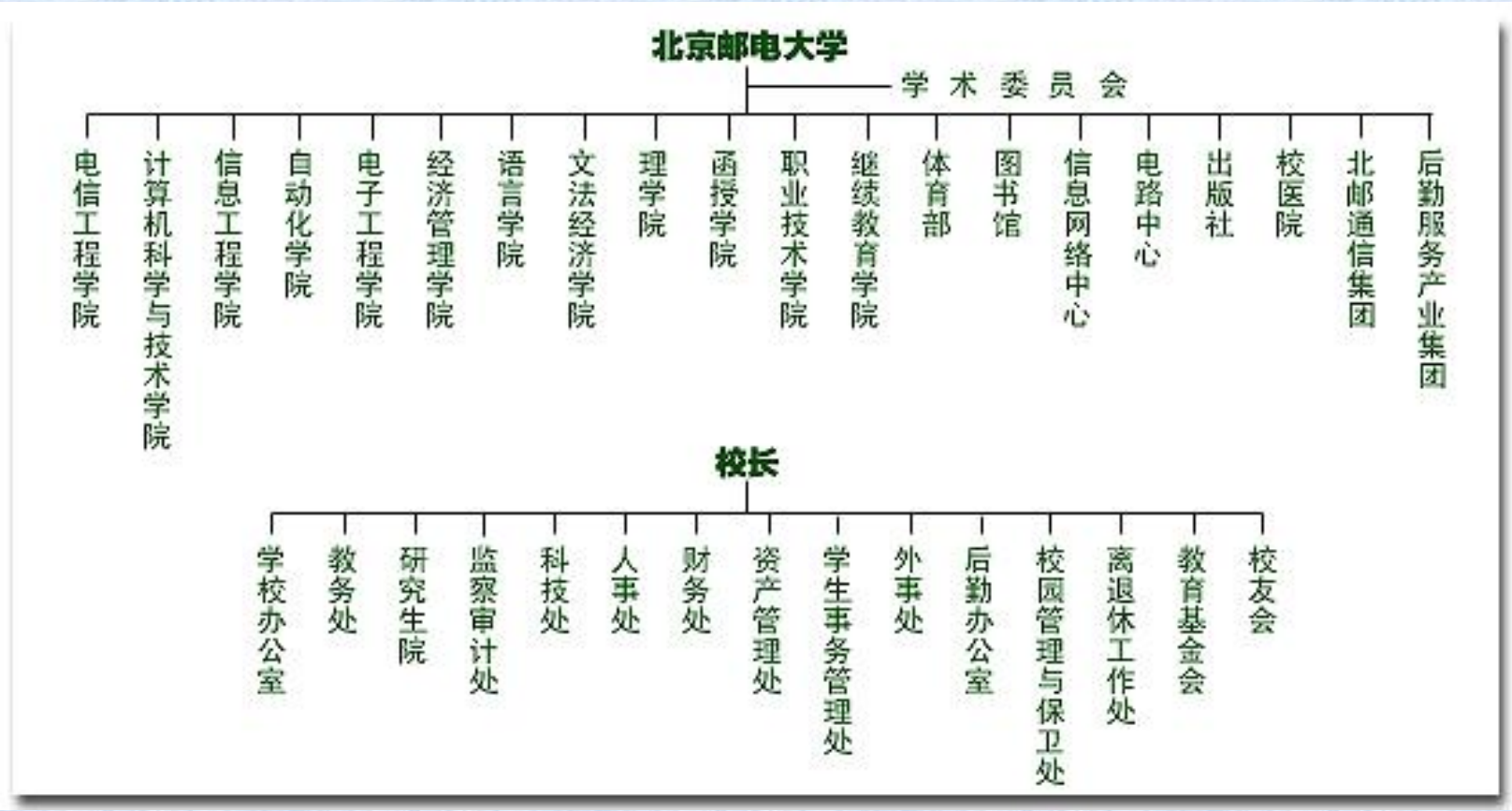
异丁烷



$$v_1 v_2 / v_3 + v_4 (v_5 - v_6 / v_7)$$



家族树,图书十进制分类,一个组织结构的层次体系,各种运算表达式,判断树.

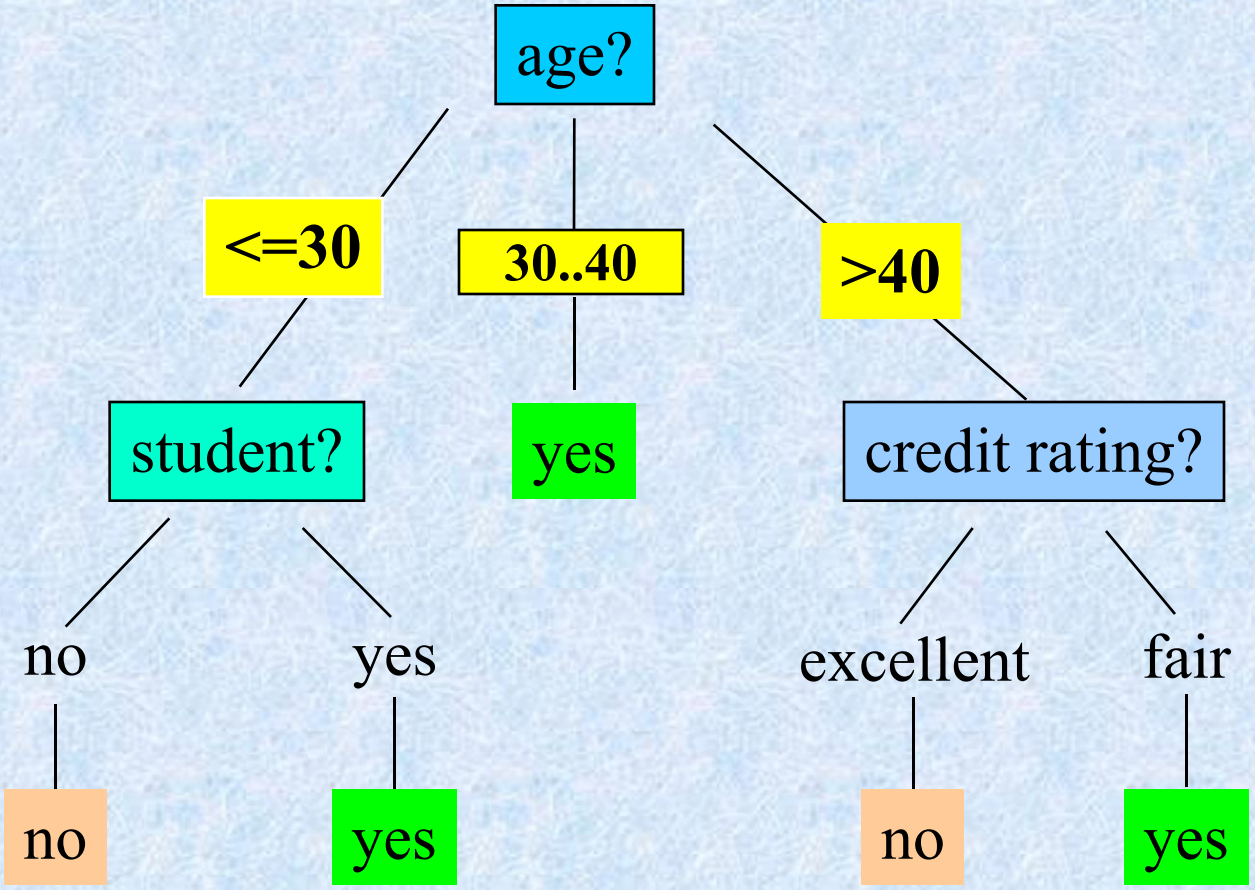

北京邮电大学
学术委员会
后勤服务产业集团
北邮通信集团
校医院
出版社
电路中心
信息网络中心
图书馆
体育部
继续教育学院
职业技术学院
函授学院
理学院
文法经济学院
语言学院
经济管理学院
电子工程学院
自动化学院
信息工程学院
计算机科学与技术学院
电信工程学院
校长
校友会
教育基金会
离退休工作处
校园管理与保卫处
后勤办公室
外事处
学生事务管理处
资产管理处
财务处
人事处
科技处
监察审计处
研究生院
教务处
学校办公室

Training Dataset

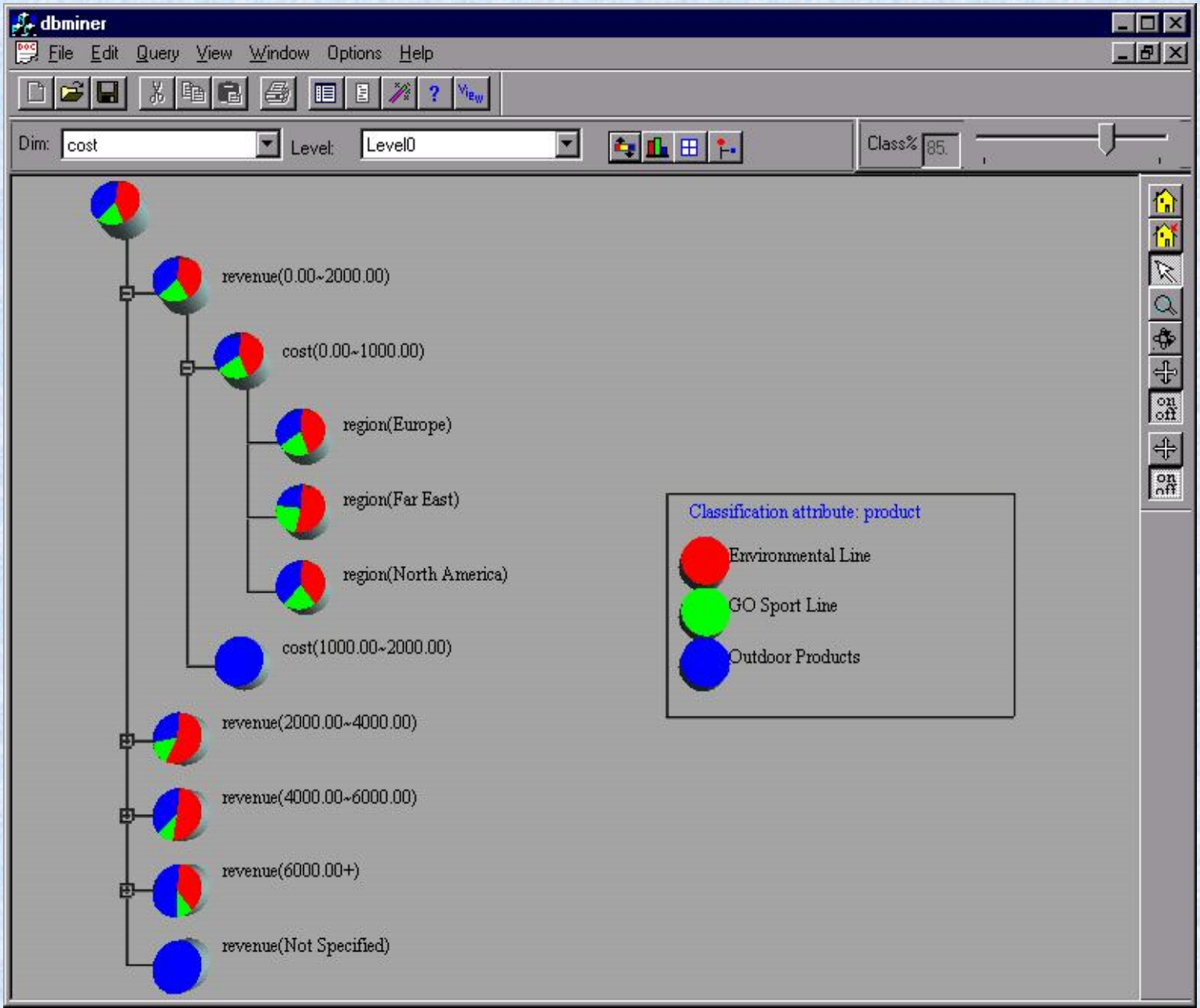
This follows an example from Quinlan's ID3

<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

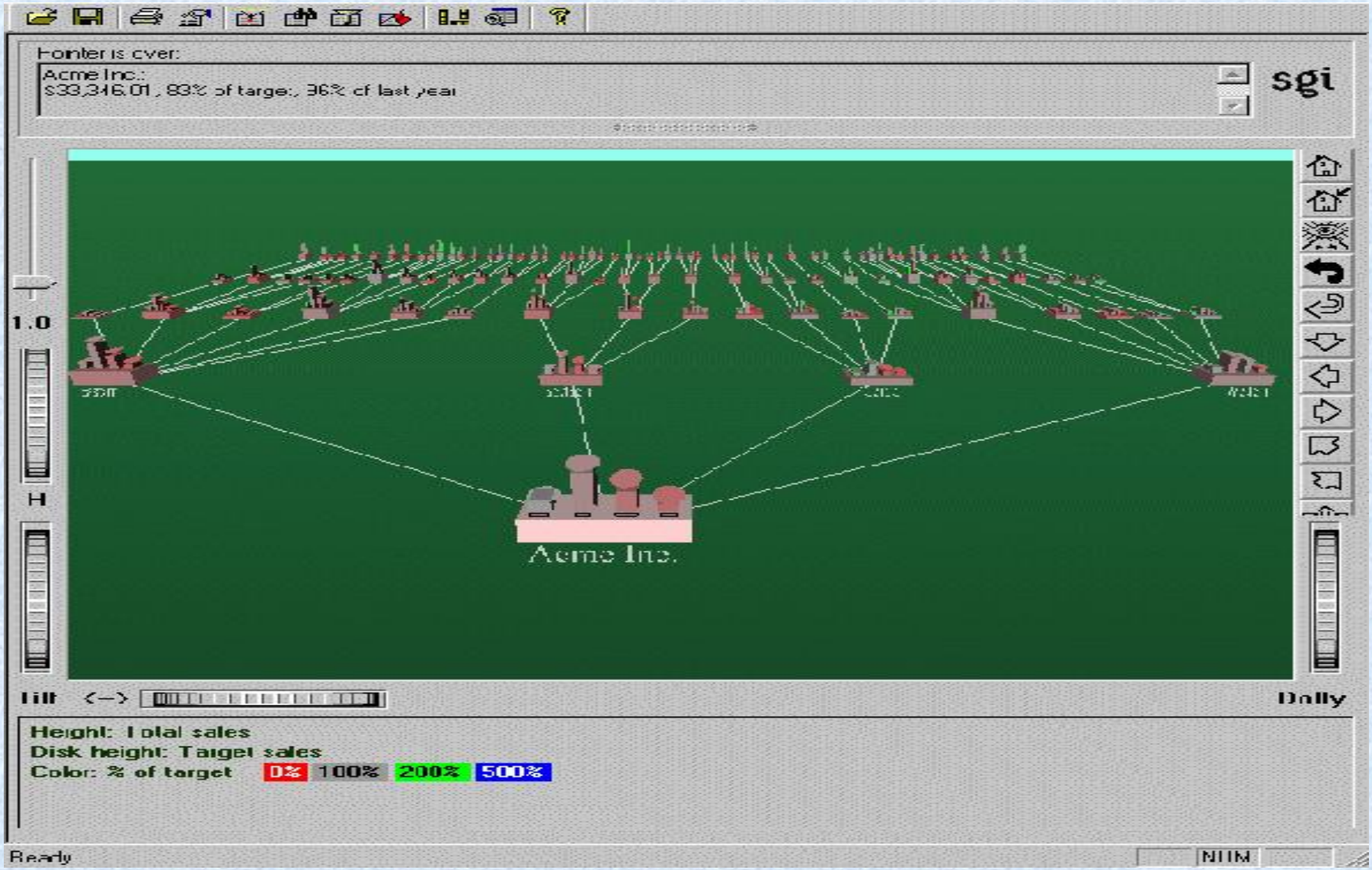
Output: A Decision Tree for "*buys_computer*"



Presentation of Classification Results



Visualization of a Decision Tree in SGI/MineSet 3.0



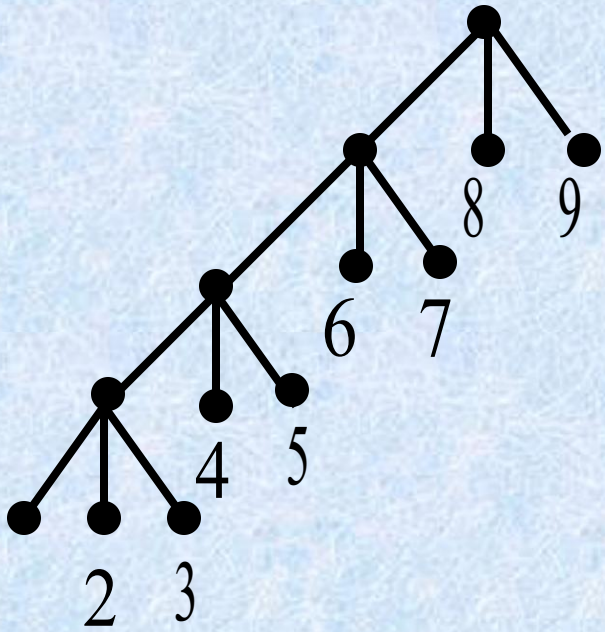
Some Tree Theorems

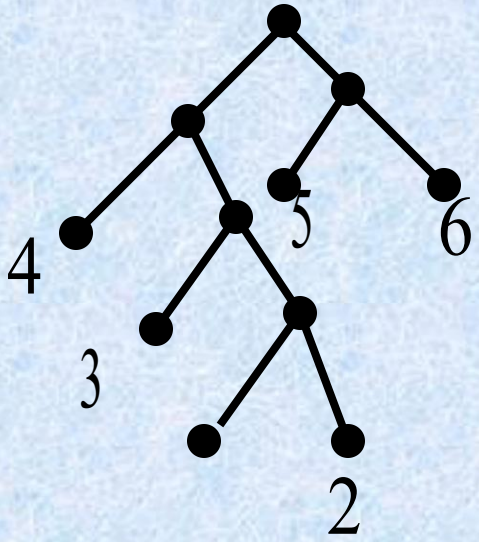
- A tree with n nodes has $n-1$ edges.
- A full m-ary tree with i internal nodes has $n=mi+1$ nodes, and $\ell=(m-1)i+1$ leaves.
  - Proof: There are mi children of internal nodes, plus the root. And, $\ell = n-i = (m-1)i+1$. □
  - Thus, given m, we can compute any of i, n, and ℓ from any of the others.

More Theorems

- **Definition:** The *level* of a node is the length of the simple path from the root to the node.
  - The *height* of a tree is maximum node level.
  - A rooted *m*-ary tree with height *h* is *balanced* if all leaves are at levels *h* or *h*−1.
- **Theorem:** There are at most m^h leaves in an *m*-ary tree of height *h*.
  - **Corollary:** An *m*-ary tree with ℓ leaves has height $h \geq \lceil \log_m \ell \rceil$. If *m* is full and balanced then $h = \lceil \log_m \ell \rceil$

定理 设有完全m叉树，其树叶leaves为t,分支点nonleaves数为i,则$(m-1)i=t-1$





例 28盏灯拟用一个电源插座,问需要多少块四插座接线板?

$$(m-1)i=t-1$$

$$(4-1)i=28-1 \quad i=9$$

需要9个接线板.

[定理]： 若T为m元树，则（m－1）i≥t－1，其中i为枝点数，t为叶数。

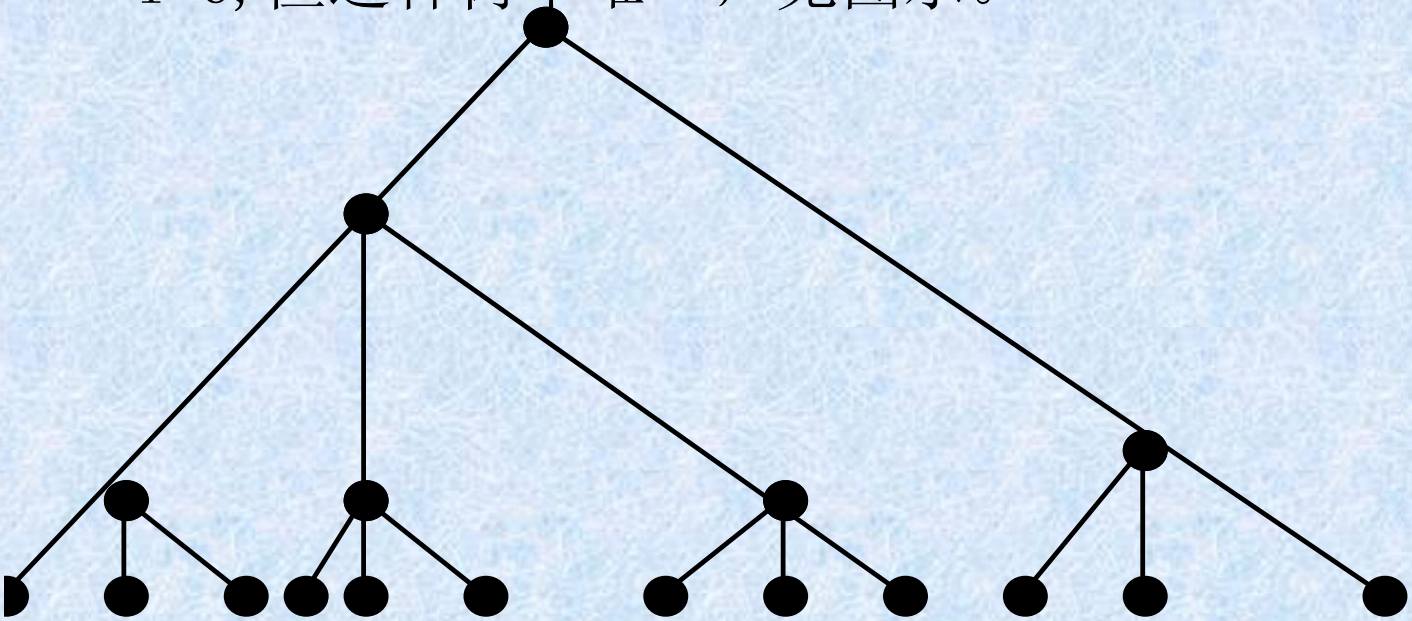
例：一台三地址计算机，用一个加法指令可求三个数之和，现要计算１２个数之和，至少要几次加法指令？

解：　　（m－１）i ≥ t－１

（３－１）i ≥１２－１＝１１

i ≥11/2＝５．５

i=6,但这种树不唯一，见图示。



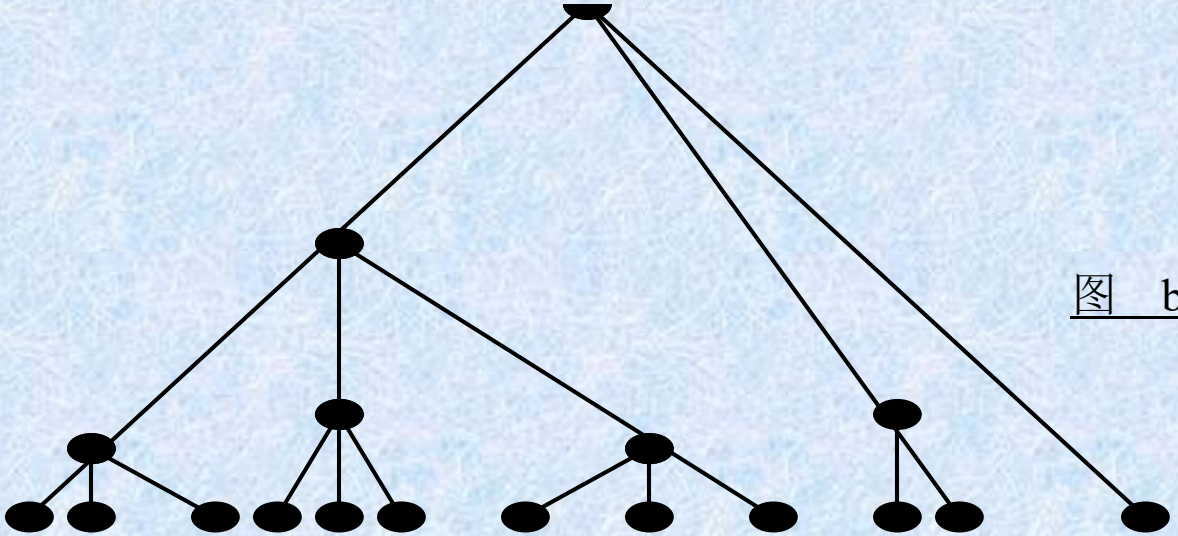


图 b

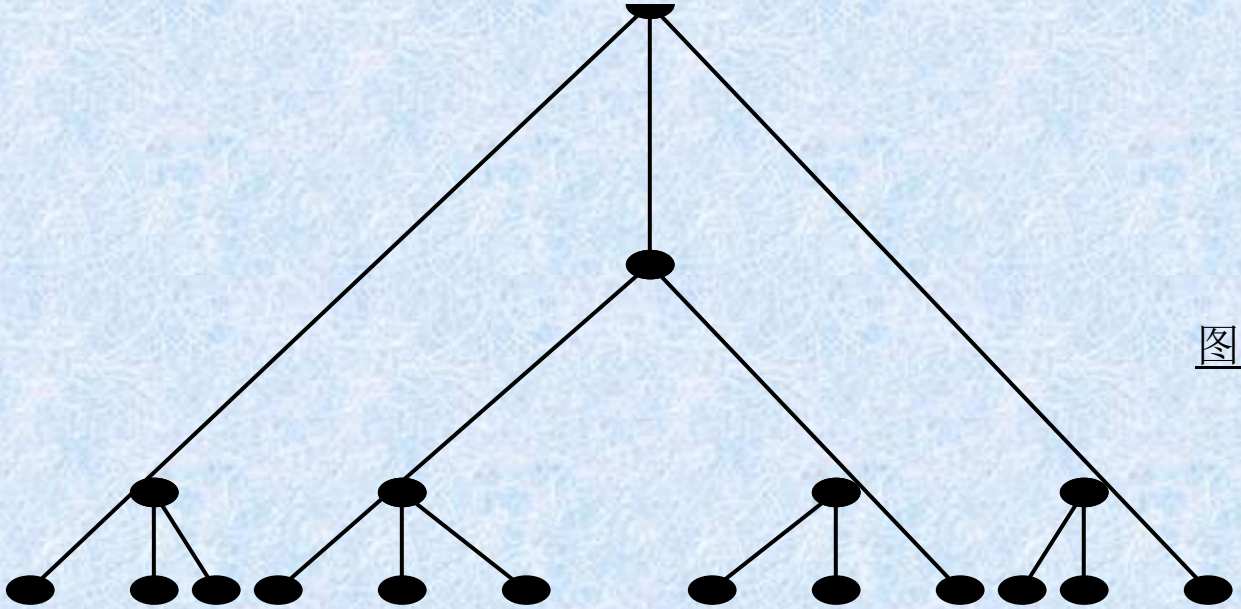


图 c

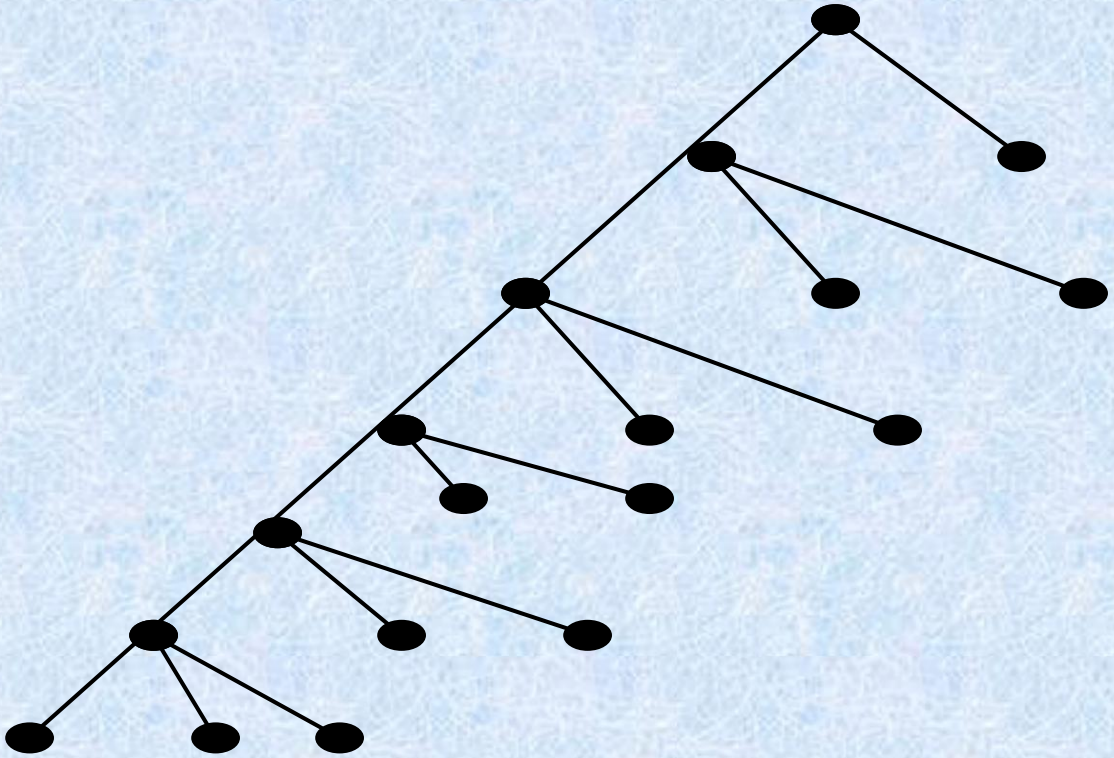


图 d

balanced

- A rooted m-ary tree of height *h* is **balanced** if all leaves are at levels *h* or *h-1*.

§11.2: Applications of Trees

- Binary search trees
- Decision trees
  - Minimum comparisons in sorting algorithms
  - 8 硬币问题
- Prefix codes
  - Huffman coding
- Game trees

Binary search trees

- Form a binary tree for the words:

Mathematics,physics,geography,zoology,meteorology,geology,psychology,and chemistry (using alphabetical order)

Mathematics,physics,geography,zoology,meteorology,geology,psychology,and chemistry

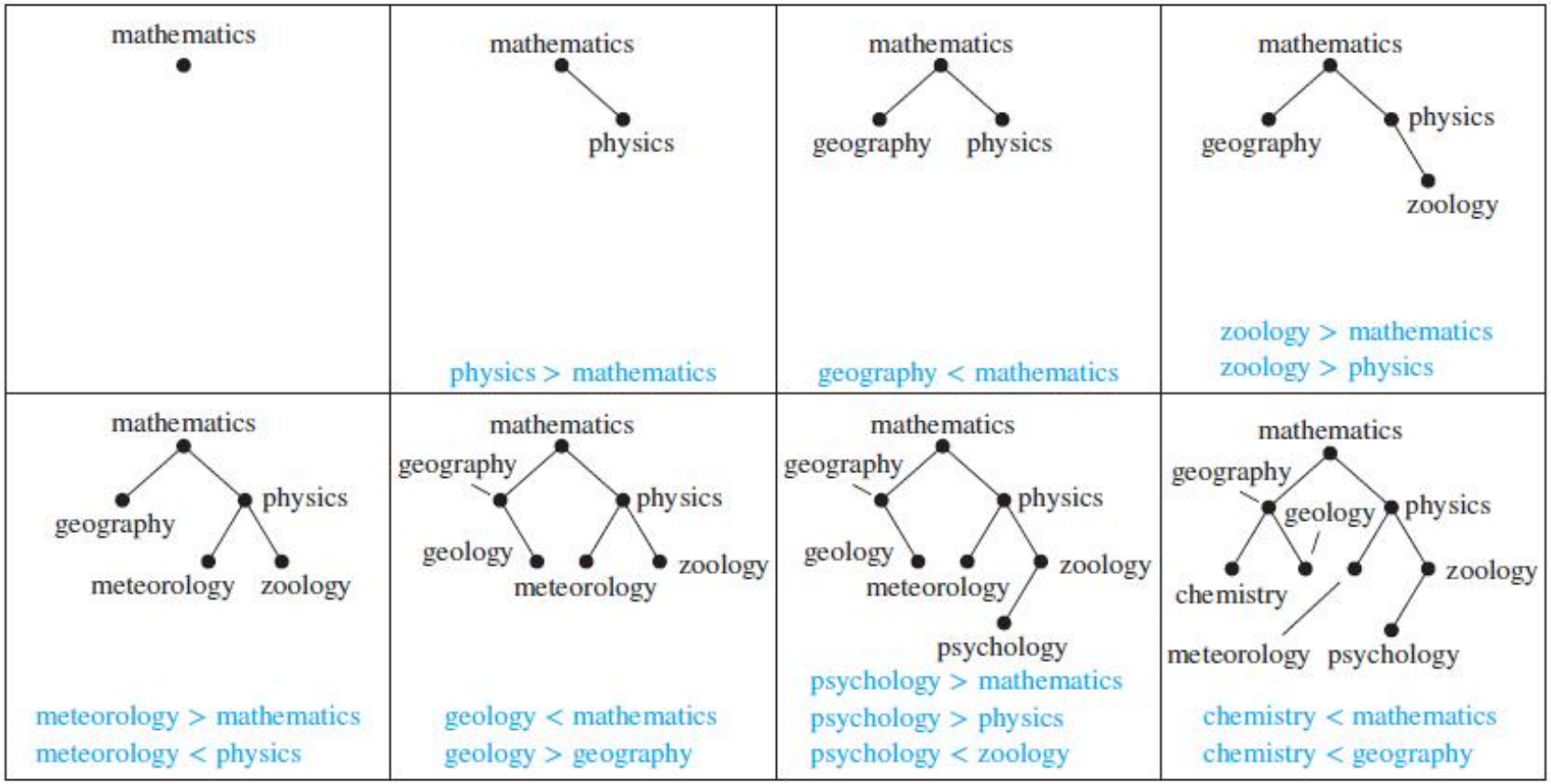


FIGURE 1 **Constructing a binary search tree.**

Decision trees

- Minimum comparisons in sorting algorithms
- 8 硬币问题

Minimum comparisons in sorting algorithms

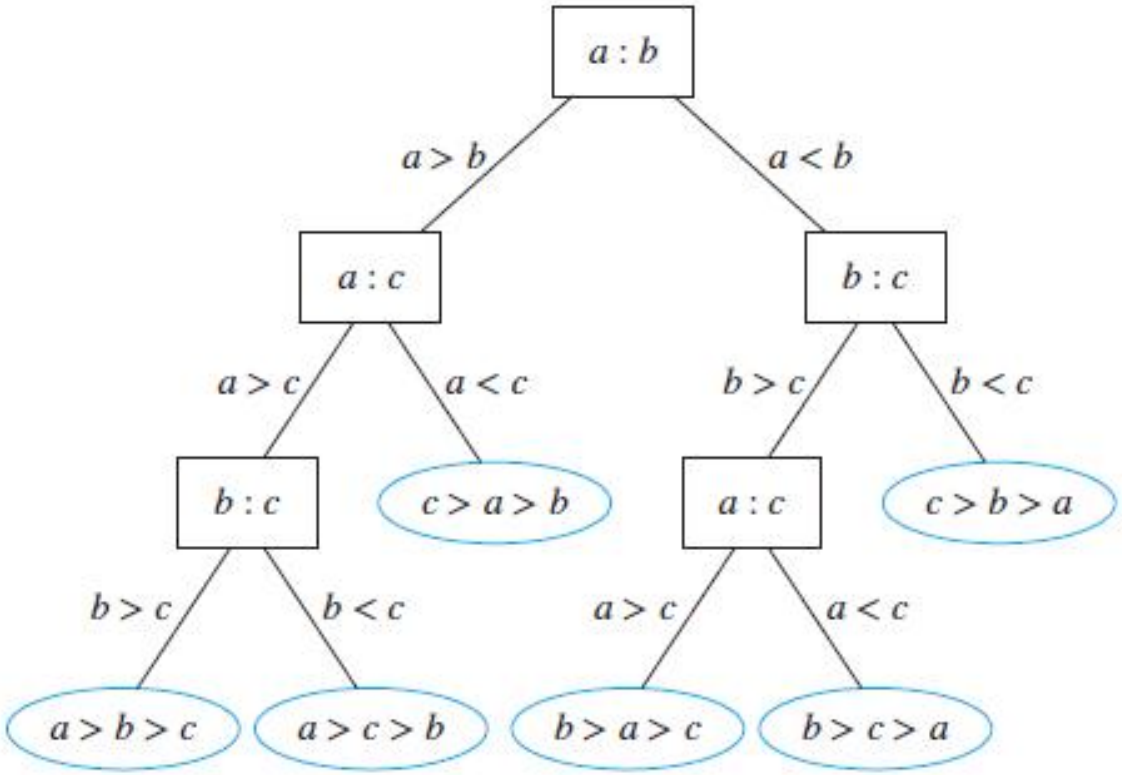


FIGURE 4 **A decision tree for sorting three distinct elements.**

- The average number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log n)$.

- Because the height of a binary tree with $n!$ leaves is at least $\lceil \log_2 n! \rceil$, at least $\lceil \log_2 n! \rceil$ comparisons are needed $\lceil \log n! \rceil$ is $\Theta(n \log n)$.

8 硬币问题

- Suppose there are seven coins, all with the same weight, and a counterfeit coin that weighs less than the others. How many weighings are necessary using a balance scale to determine which of the eight coins is the counterfeit one? Give an algorithm for finding this counterfeit coin.

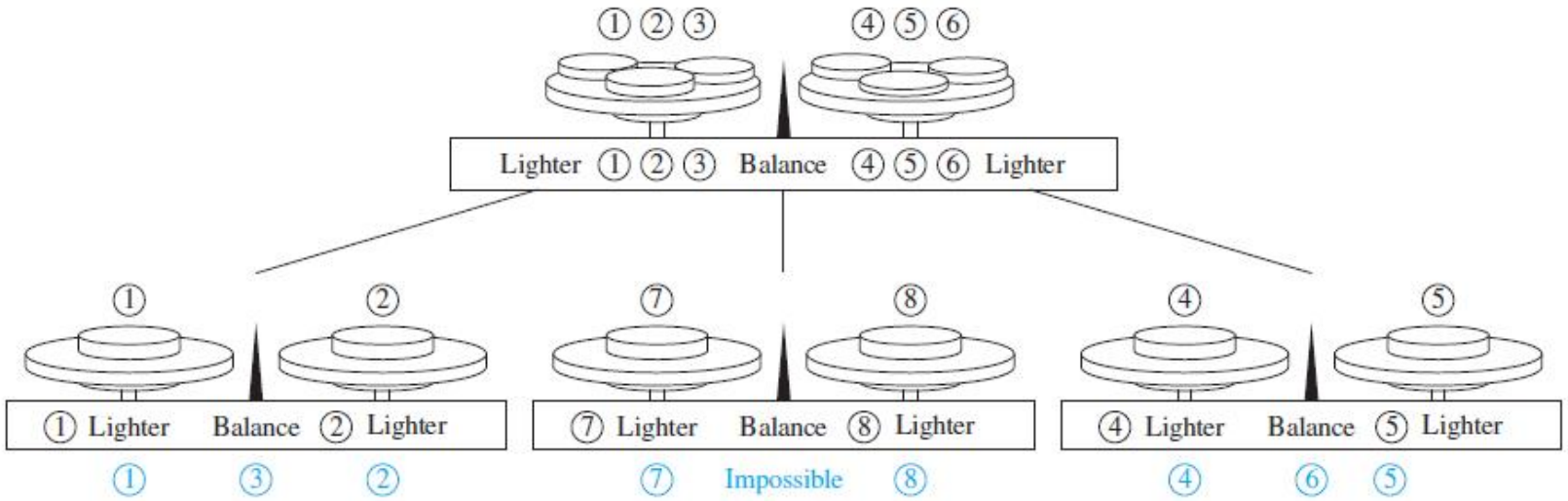


FIGURE 3 **A decision tree for locating a counterfeit coin. The counterfeit coin is shown in color below each final weighing.**

Prefix Codes

- Huffman Coding

前缀码/Prefix Codes

1. 计算机是用 0 和 1 序列来表示和存储信息。通讯编码也是这样处理的。

如２６个英文字母(a-z)，用v位则可表示 2^v，由于 $2^4=16<26<2^5=32$，故要用 5 位二进制才能区分２６个字母。

2. 是否能用不同位数的二进制数来表示信息，尽量缩短通讯时序列的长度。这是可行的，但要注意:如００表示Ａ，０１表示Ｂ，０００１表示ｚ，那么收到０００１是代表ＡＢ还是ｚ？怎样避免二义性？

[定义]前缀： a， b， c 均为二进制序列，如果 b ＝ a 并列 c， c 不是空序列，则称 a 是 b 的前缀/prefix。即 a 是 b 的前面一部分。

[定义] 前缀码：一个二进制序列集合，如果这些二进制序列互不为前缀，则称此集合为前缀码/prefix code。

例：二元有序加权树

每枝点左边对应权为0，右边对应权1，叶对应一个码（即每片叶都有一条从根到叶的路径，路径上的权排成序即为叶的码，也称作叶的名。

二元前缀码

设 $\alpha_1,\alpha_2,\cdots,\alpha_{n-1},\alpha_n$ 是长为 n 的符号串，

$\alpha_1,\alpha_1\alpha_2,\cdots,\alpha_1\alpha_2\cdots\alpha_{n-1}$

均为该符号串的前缀，它们的长度分别为 1,2, ，n -1。

$A=\{\beta_1,\beta_2,\cdots,\beta_m\}$为一个符号串集合,对于任意的 $\beta_i,\beta_j\in A$ $(i\neq j),\beta_i$与 β_j 互不为前缀,则称 A 为前缀码。

若符号串 β_i $(i=1,2,\cdots,m)$ 中只出现0,1 两个符号, 则称 A 为二元前缀码。

$B_1 = \{0,10,110,1111\}$ √

$B_2 = \{1,01,001,000\}$ √

$B_3 = \{1,11,101,001,0011\}$ ×

$B_4 = \{b,c,aa,ac,aba,abb,abc\}$ √

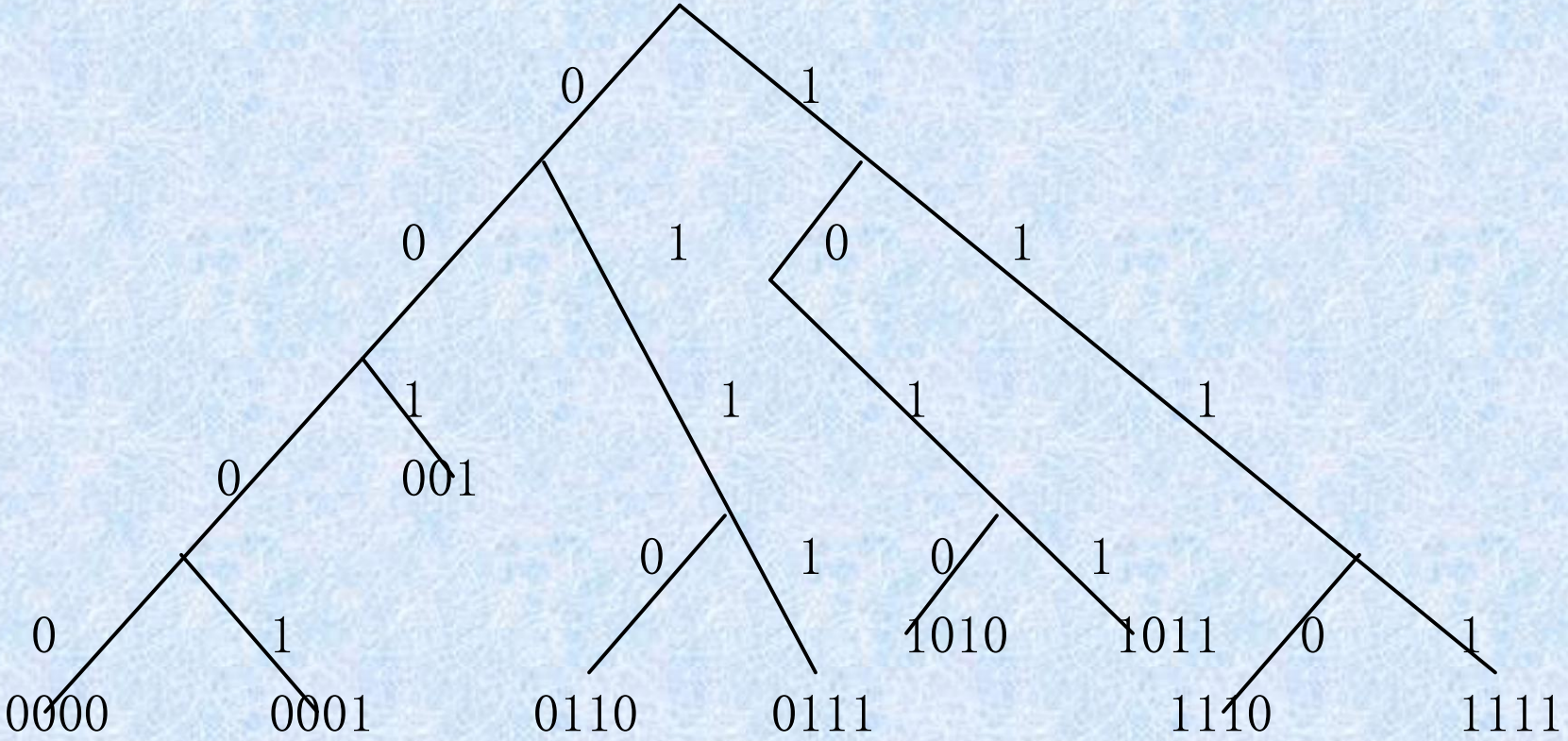
$B_5 = \{b,c,a,aa,ac,aba,abb,abc\}$ ×

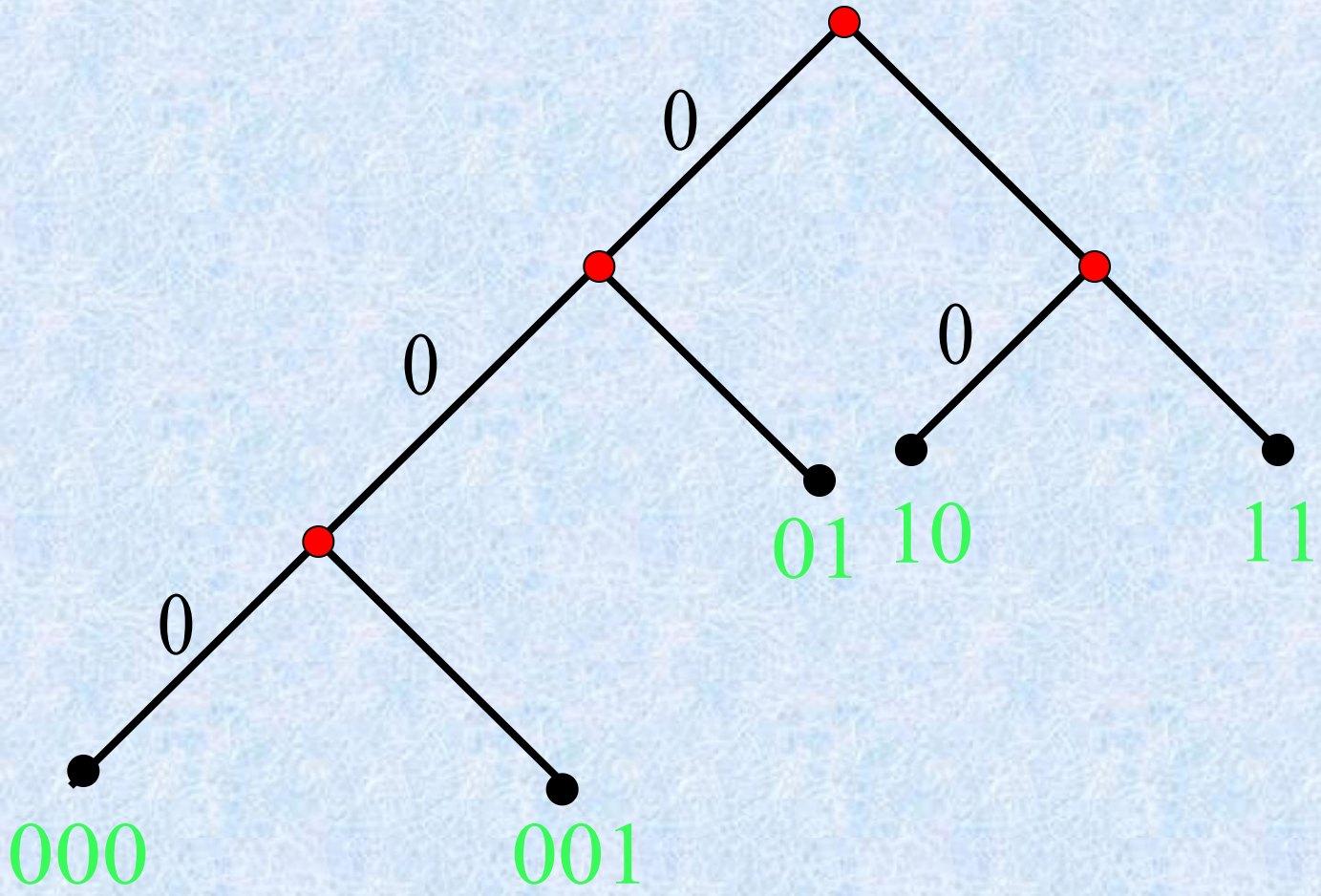
定理

任意一棵二叉树都可产生一个前缀码。

定理

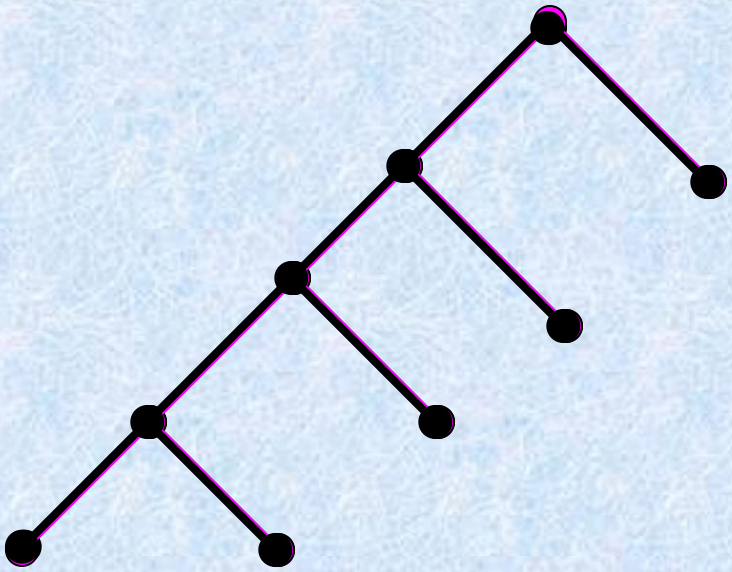
任何一个前缀码都对应一棵二叉树。


0
1
0
1
0
1
1
1
1
1
0
001
0
1
0
1
0
1
0
1
0000
0001
0110
0111
1010
1011
1110
1111



{01,10,11,001,000}

{1,01,001,0000,0001}



由于枝点才可能是叶的前缀，而枝点不对应一个码，故这样一棵树所有叶对应的码构成前缀码。

二元完全树的叶只有 2^h 片（ h 为树高），用二元树的前缀码，最长码长是树高 h ，最多不超过 2^h 片。

如上例 2 6 个字母，就可构造树高为5的二元树，然后用相应的前缀码分别表示各个字母，将常用的字母用较短的编码表示，可缩短整个通讯序列的长度，提高效率，即：

设 l_i 为第 i 个字母前缀码长，w_i 为此字母在

1 0 0 0 个字母中出现次数（频率,概率），$\sum_{i=1}^{26} l_i w_i$

就是 1 0 0 0 个字母所需二进制代码总长。

目标:构造一个适当的二元有序加权树，使总长达到最小。(对任何标识符及文字的集合均可采用).

[定义]树的权：设树 T 有 t 片叶，对应的权为 w_1，w_2，…，w_t（叶以权 w_i 命名），根到叶 w_i 的道路长为 $l(w_i) = l_i$，则称 T 为关于权 w_1，w_2，…，

w_t 的二元树，$w(T) = \sum_{i=1}^{t} w_i l_i$ 称为树 T 的权。

最优树问题

带权二叉树: 每片树叶都带权的二叉树.

带权二叉树的权:

$w(T)=\sum_{i=1}^{t} w_i L(w_i)$ 其中 $L(w_i)$ 为带权 w_i 的树叶的通路长度.

最优树: 在所有带权 w_1, $w_2, \cdots, w_t$ 的二叉树中, $w(T)$ 最小的那棵树.

定理　T为带权 $w_1 \le w_2 \le \cdots \le w_t$ 的最优树, 则

a) 带权 w_1, w_2的树叶 v_{w_1}, v_{w_2} 是兄弟.

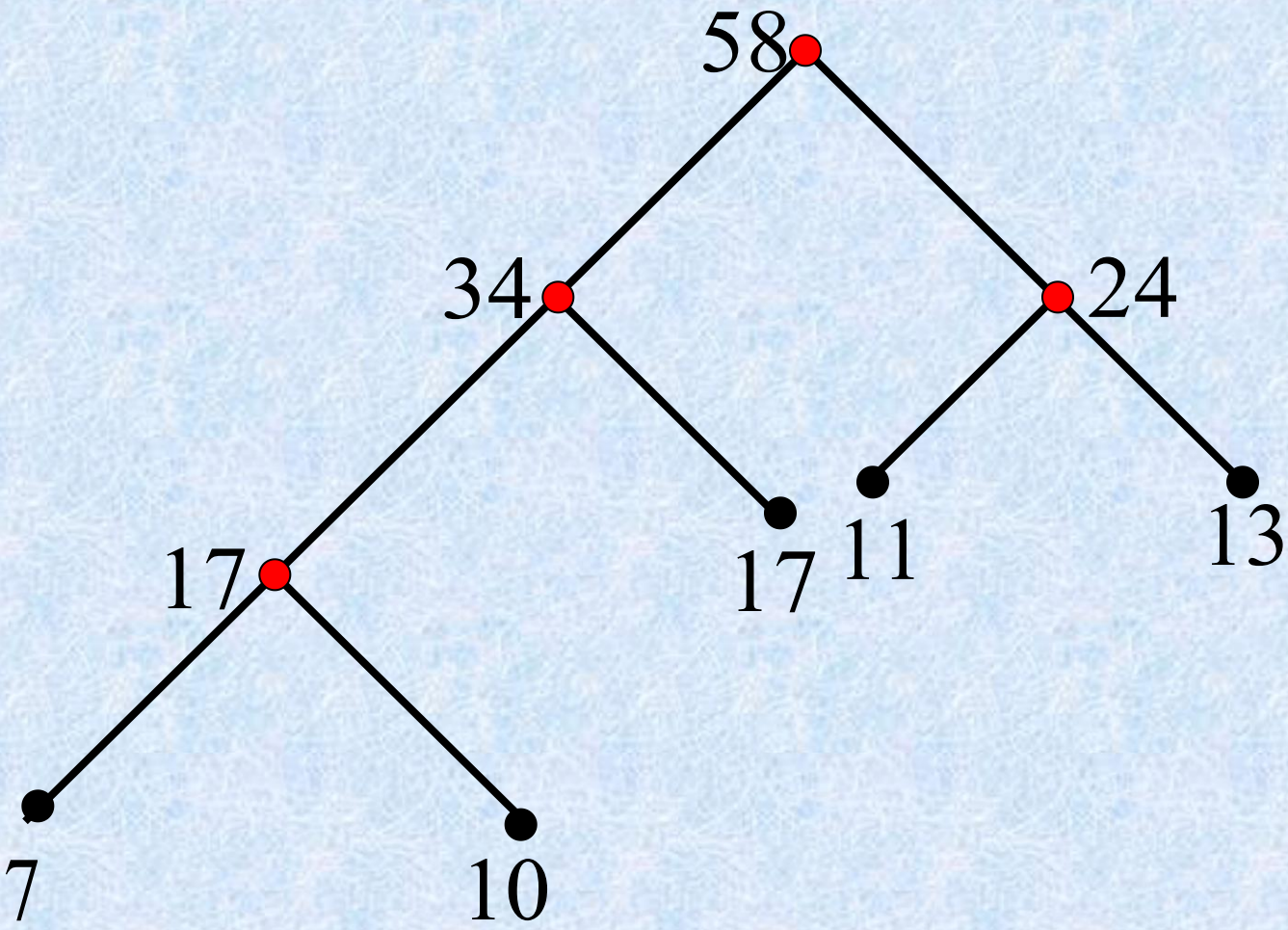
b) 以树叶 v_{w_1}, v_{w_2} 为儿子的分枝点,其通路长最长.

定理 T 为带权 $w_1 \le w_2 \le \cdots \le w_t$ 的最优树,若将以带权 w_1, w_2 的树叶为儿子的分枝点该为带权 $w_1 + w_2$ 的树叶得到一新树 T',也是最优树.

例题

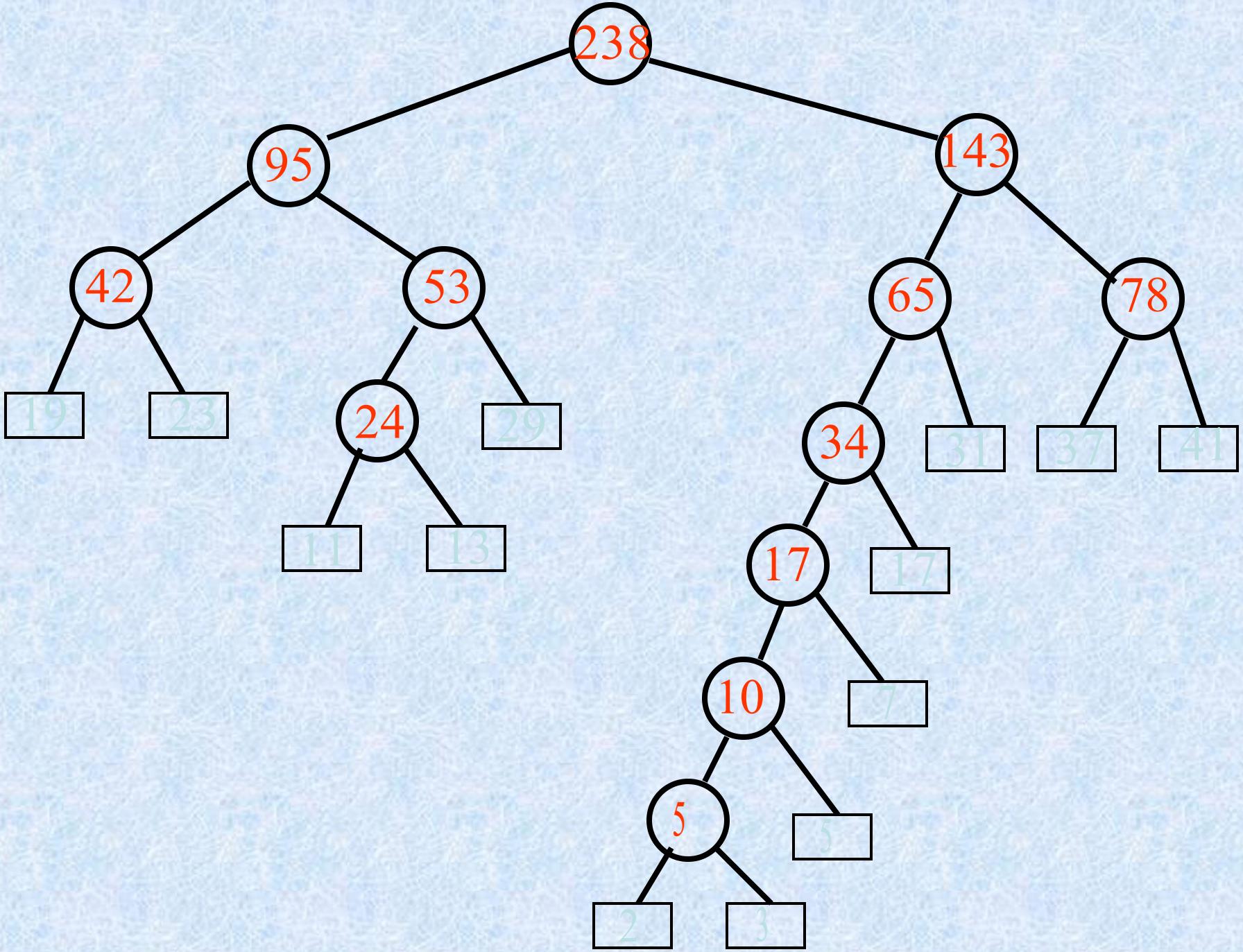
求带权7,10,11,13,17的最优树.

解 求解过程如下


58
34
24
17
17
11
13
7
10

例

有权 2,3,5,7,11,13,17,19,23 29,31,37,41求相应的最优树.


238
95
143
42
53
65
78
19
23
24
29
34
31
37
41
11
13
17
17
10
7
5
5
2
3

例

已知字母A，B，C，D，E，F，G出现的频率如下：

A—35％　B—20％　C—15％ D—10％

E –10％　F –5％　G –5％

1) 求带权 5,5,10,10,15,20,35 的最优二叉树。

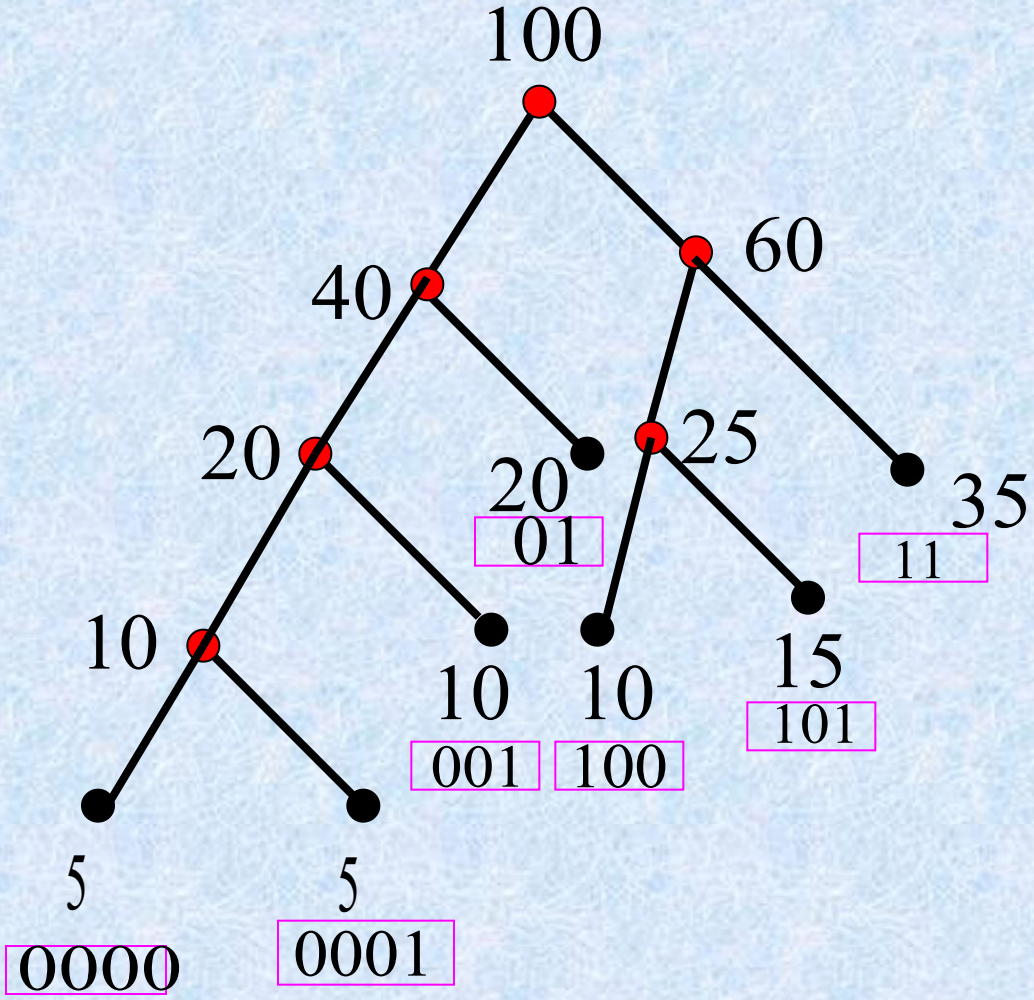
2)求T所对应的前缀码

3)设树叶 v_i 带权 $w_i \times 100$

求 v_i 处的符号串表示出

现频率为 $w\%$ 的字母

1)



2)

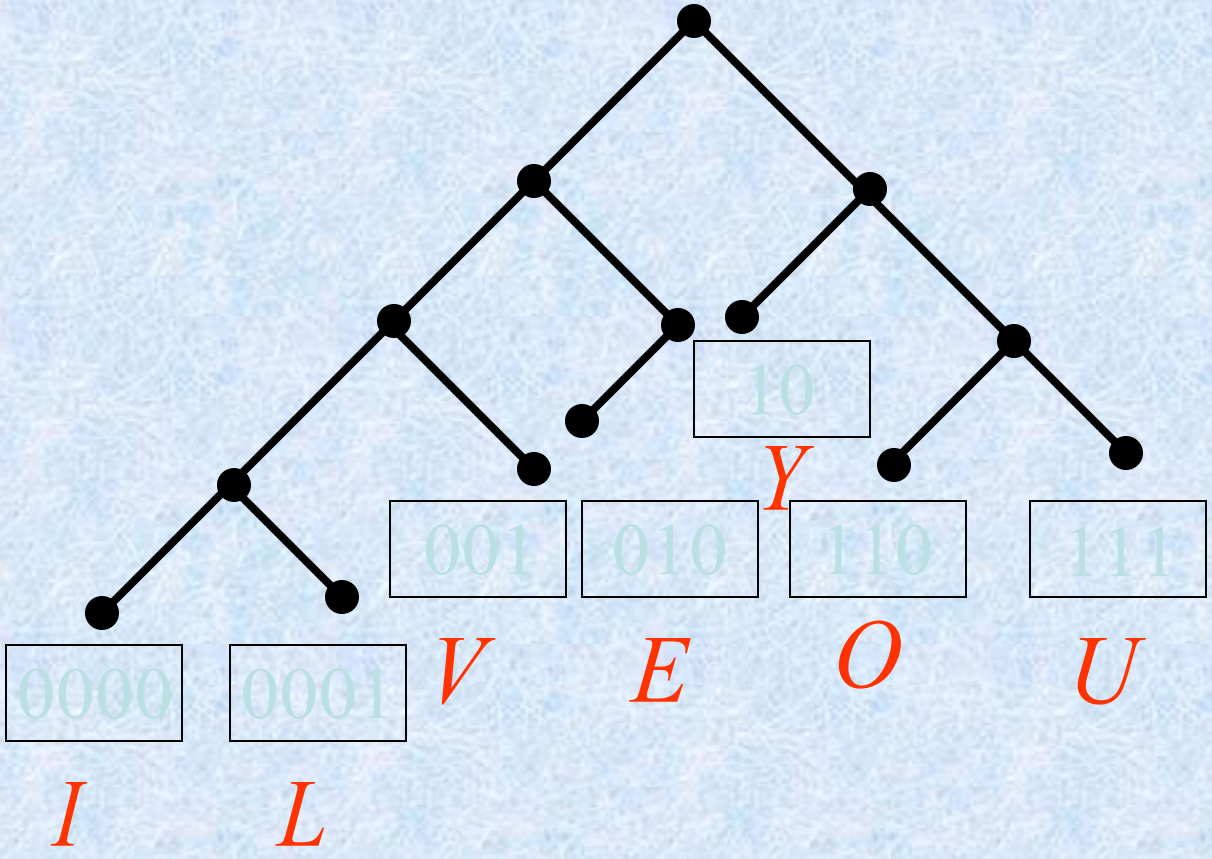
$A = \{01,11,001,100,101,0000,0001\}$

3)

11 A; 01 B; 101 C;

100 D; 001 E;

0001 F; 0000 G



11100 *a*

000 *h*

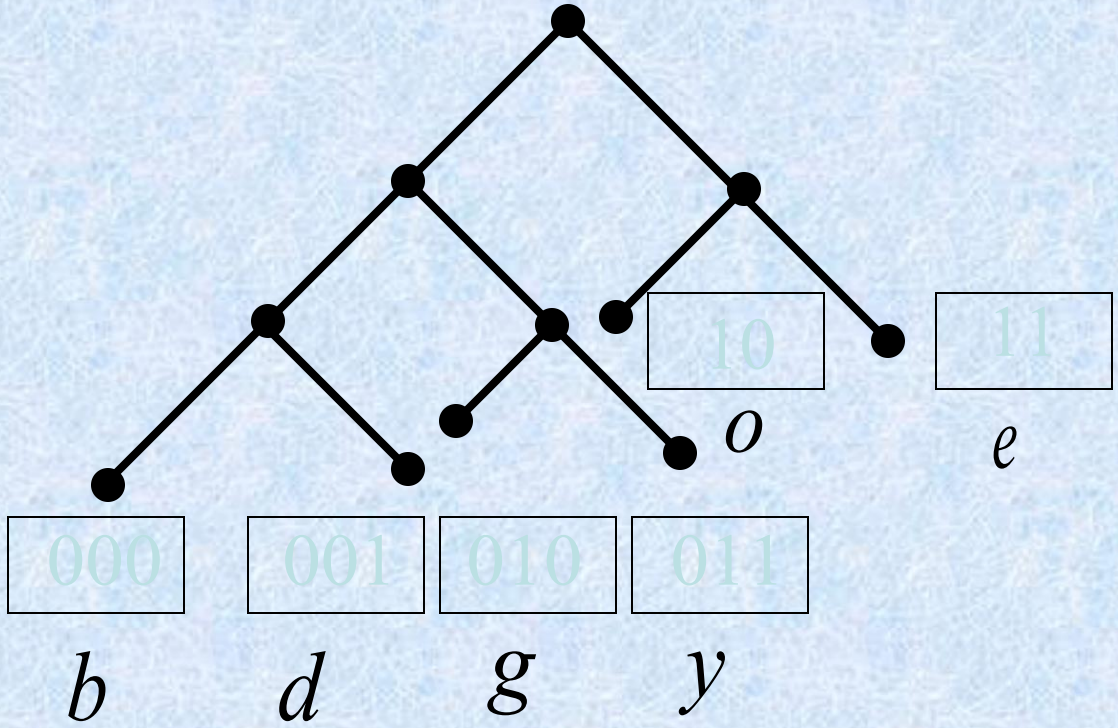
01

00000001110001010101101111

I *o* *v* *e* *y* *o* *u*

00000001110001010101101111

I *h* *a* *e* *y* *o* *u*



0101010001000011 11

G o o d b y e

Good bye !!!!!

[定义]最优树：对于确定的权w_1，w_2，…，w_t，若T中取到合适的l_1，l_2，…，l_t，使w（T）达到最小，则称T为关于权w_1，w_2，…，w_t的最优树。

注：如果对w_1，w_2，…，w_t作出一棵最优的二元有序加权树，则从根到叶的路径上的权序列集合就是前缀码。

Ｈｕｆｆｍａｎ．Ｄ·Ａ算法：

设$w_1 \leqslant w_2 \leqslant w_3 \ldots \leqslant w_t$为二元有序树的叶的权，

（1）ｔ＝２，关于w_1，w_2的最优树为

，是显然的。

（2）若w_1+w_2，w_3，w_4，…，w_t的最优树是Ｔ’　已求出，则用树  　代替叶w_1+w_2，即得到最优树T,(关于权w_1，w_2，…，w_t)。

例：构造关于权{2，3，4，4，5，5，7}的最优树。

解：

（1）{2，3，4，4，5，5，7}

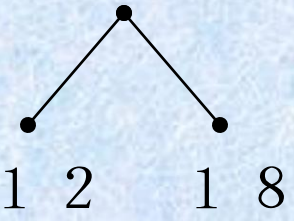
（2）{4，4，5，5，5，7}

（3）{5，5，5，7，8}

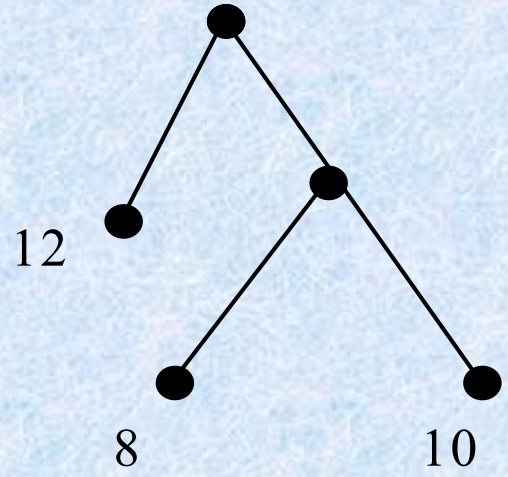
（4）{5，7，8，10}

（5）{8，10，12}

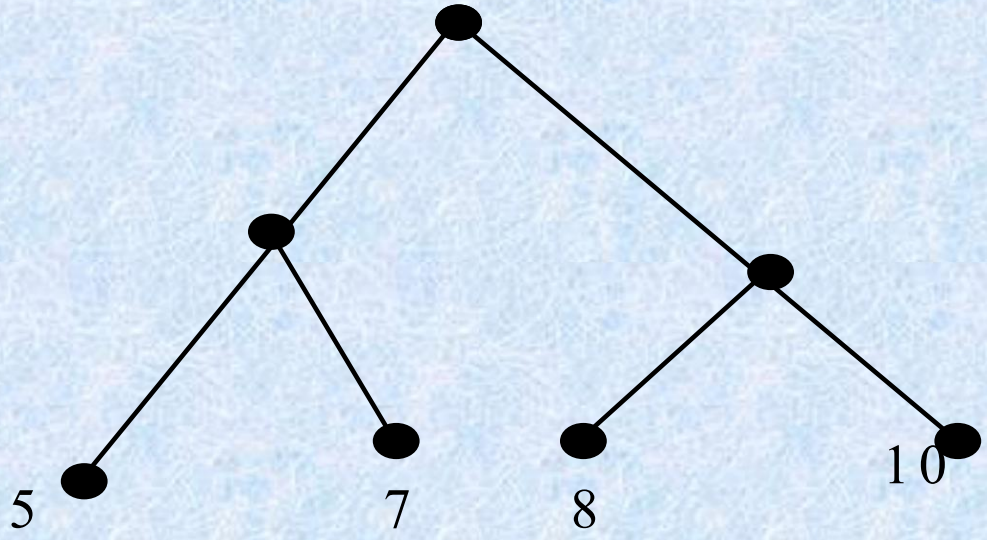
（6）{12，18}



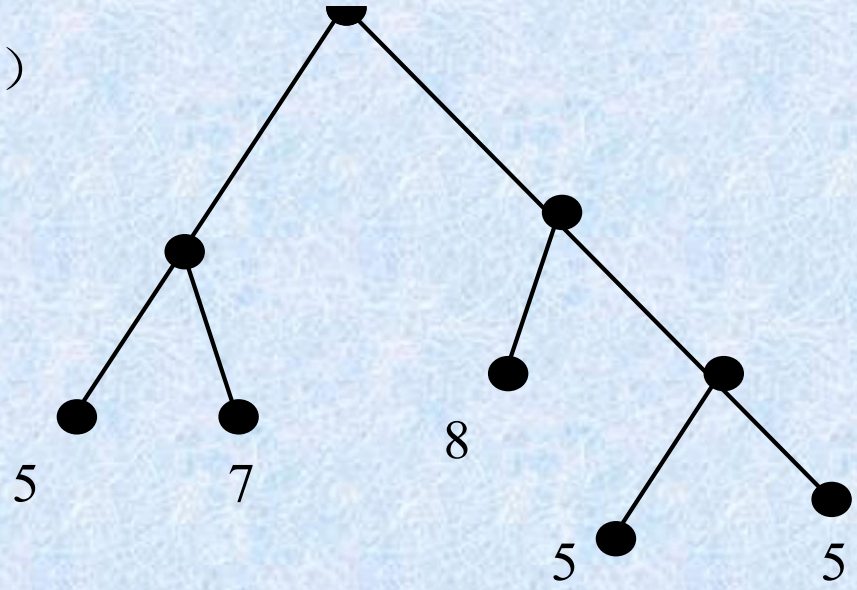
（7）倒推回去：



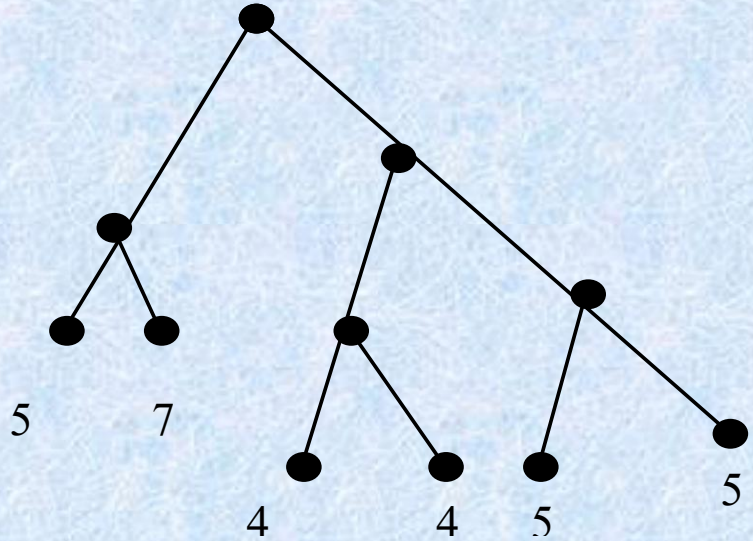
（8）

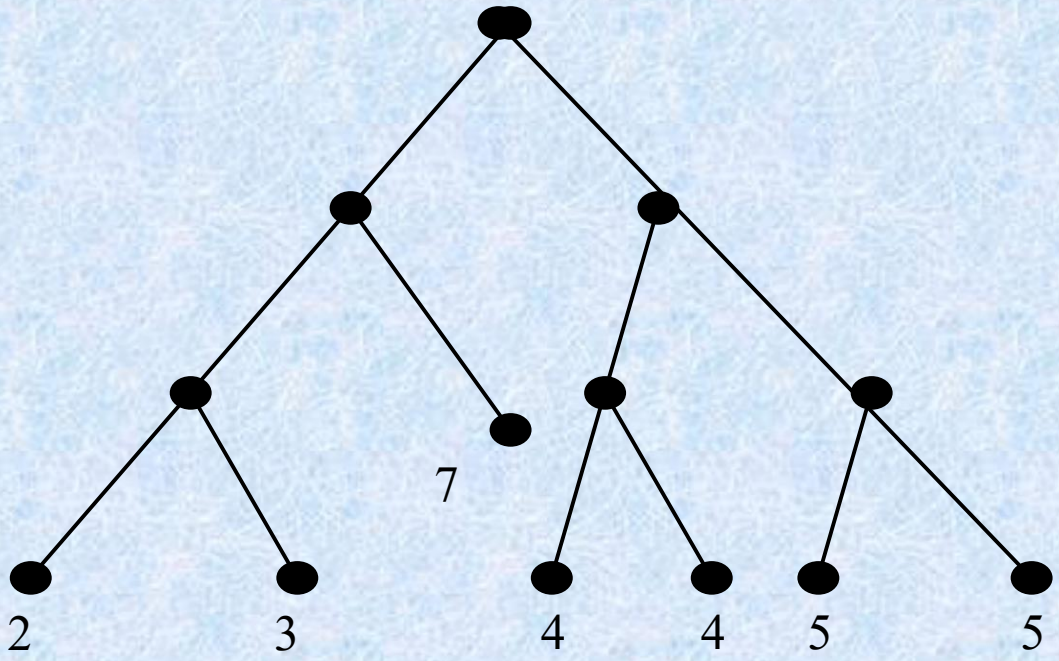


（9）



（10）





W_i	2	3	7	4	4	5	5
l_i	3	3	2	3	3	3	3

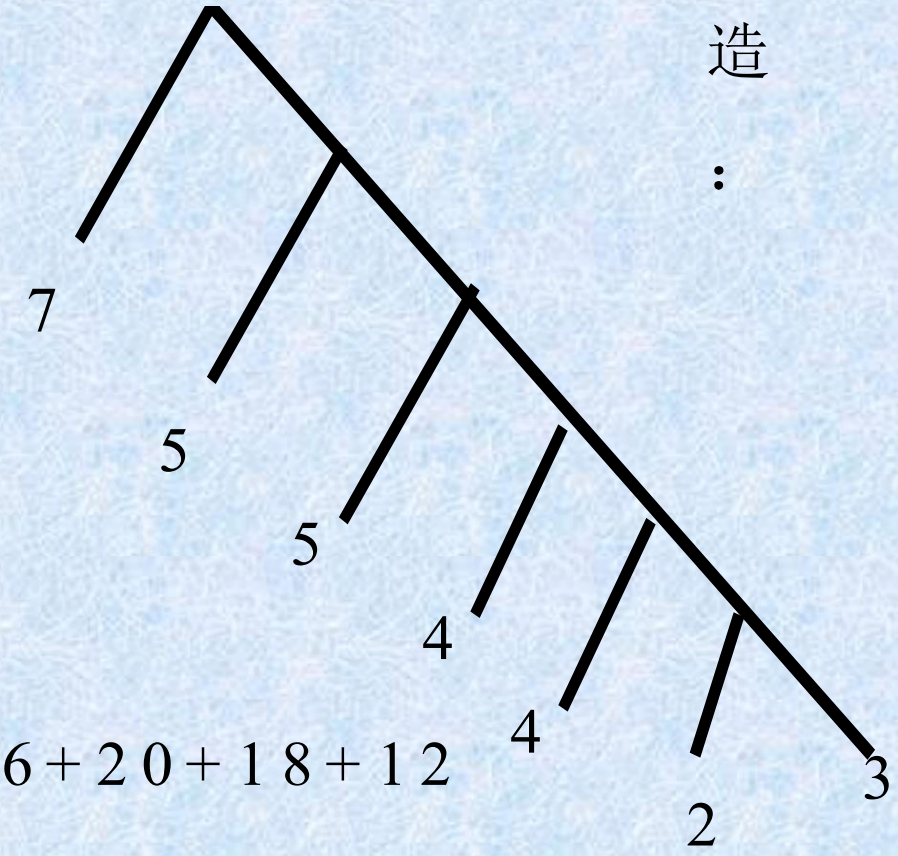
$$w(T) = \sum_{i=1}^{7} w_i l_i$$

$$= 2 \times 3 + 3 \times 3 + 7 \times 2 + 4 \times 3 + 4 \times 3 + 5 \times 3 + 5 \times 3$$

$$= 83$$

w_i	7	5	5	4	4	3	2
l_i	1	2	3	4	5	6	6

另外构造：



$$w(T) = \sum_{i=1}^{7} w_i l_i$$

$$= 7+10+15+16+20+18+12$$

$$= 98$$

作业

- §11.1 – 4, 16, 20, 28, 44
- §11.2 – 6, 16, 30

树/Trees

11.1 树的概念/Introduction of Trees

11.2 树的应用/Applications of Trees

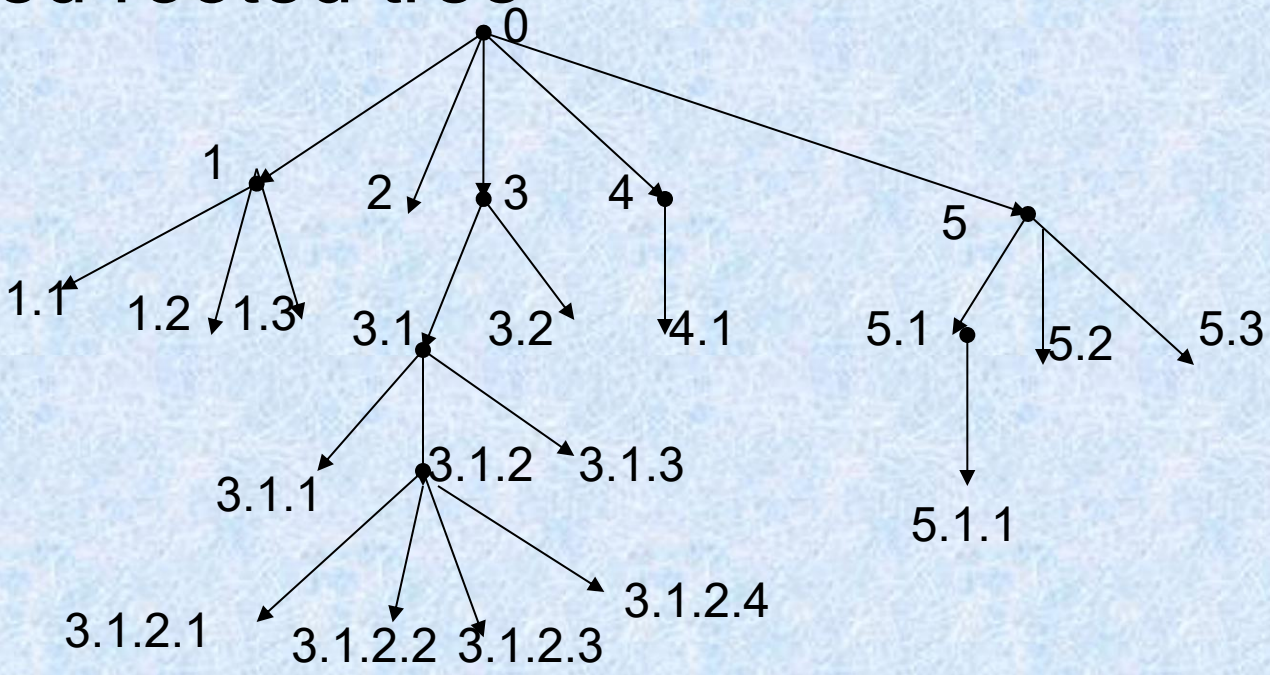
11.3 树的遍历/Tree Traversal

11.4 生成树Spanning Trees

11.5 最小生成树 minimum Spanning Trees

11.3 Tree Traversal

- Universal address systems
- ordered rooted tree

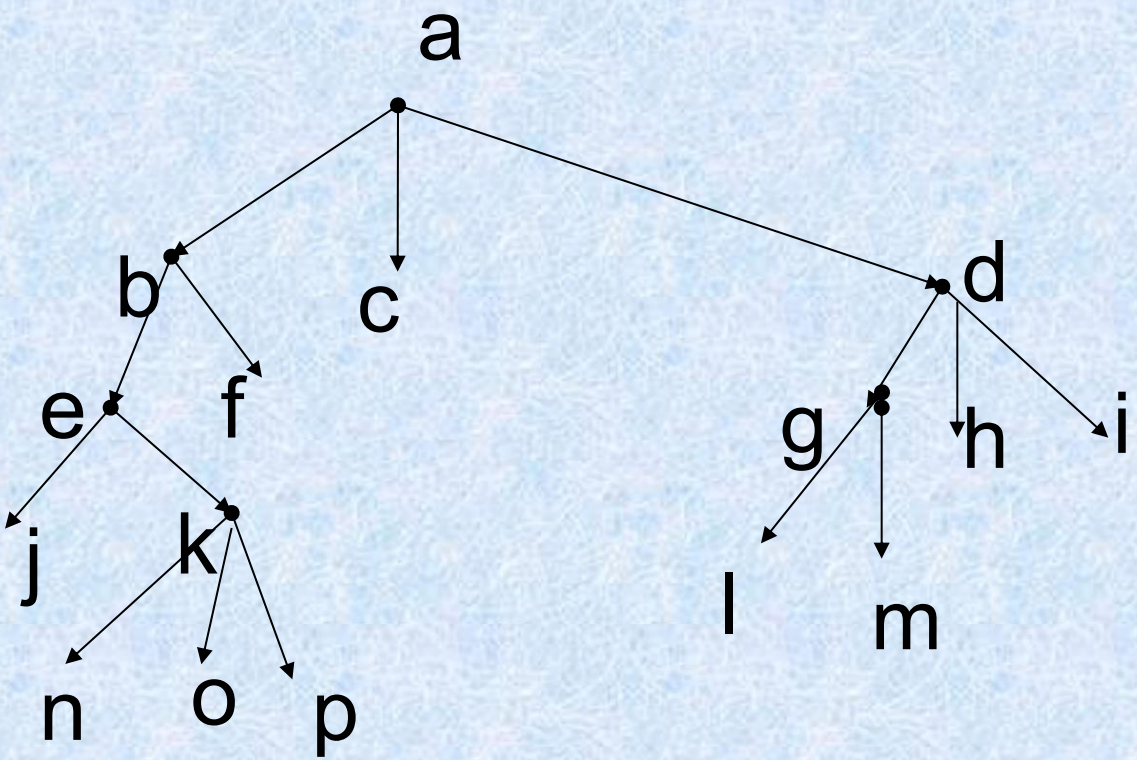


0<1<1.1<1.2<1.3<2<3<3.1<3.1.1<3.1.2<3.1.2.1<3.1.2.2<3.1.2.3<3.1.2.4<3.1.3<3.2<4<4.1<5<5.1<5.1.1<5.2<5.3

Traversal Algorithms

- Definition 1:Let *T* be an ordered rooted tree with root *r*. If T consists only of *r*, then *r* is the **preorder traversal** of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are subtrees at *r* from left to right in *T*. The preorder traversal begins by visiting r. It continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.

preorder traversal

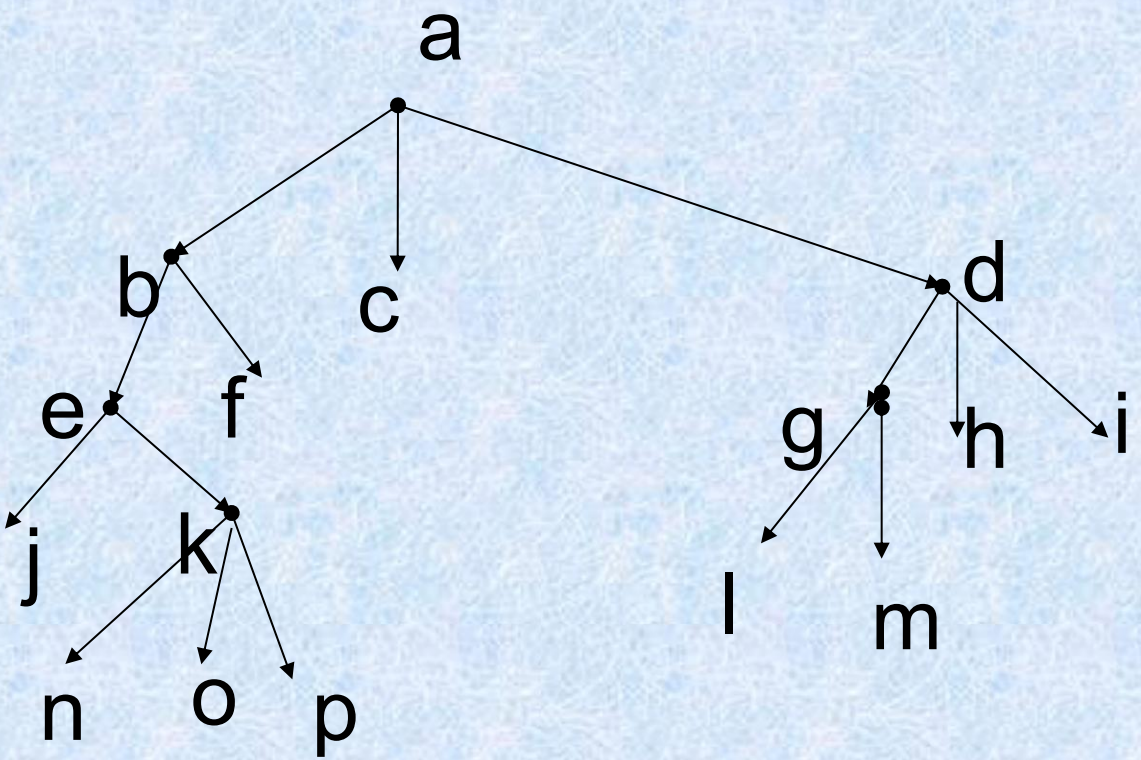


a ）前序：a,b,e,j,k,n,o,p,f, c,d,g,l,m,h,i

Traversal Algorithms

- Definition 2:Let *T* be an ordered rooted tree with root *r*. If T consists only of *r*, then *r* is the **inorder traversal** of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are subtrees at *r* from left to right in *T*. The inorder traversal begins by traversing T_1 in inorder , then visiting r. It continues by traversing T_2 in inorder, and so on, until T_n is traversed in inorder.

inorder traversal

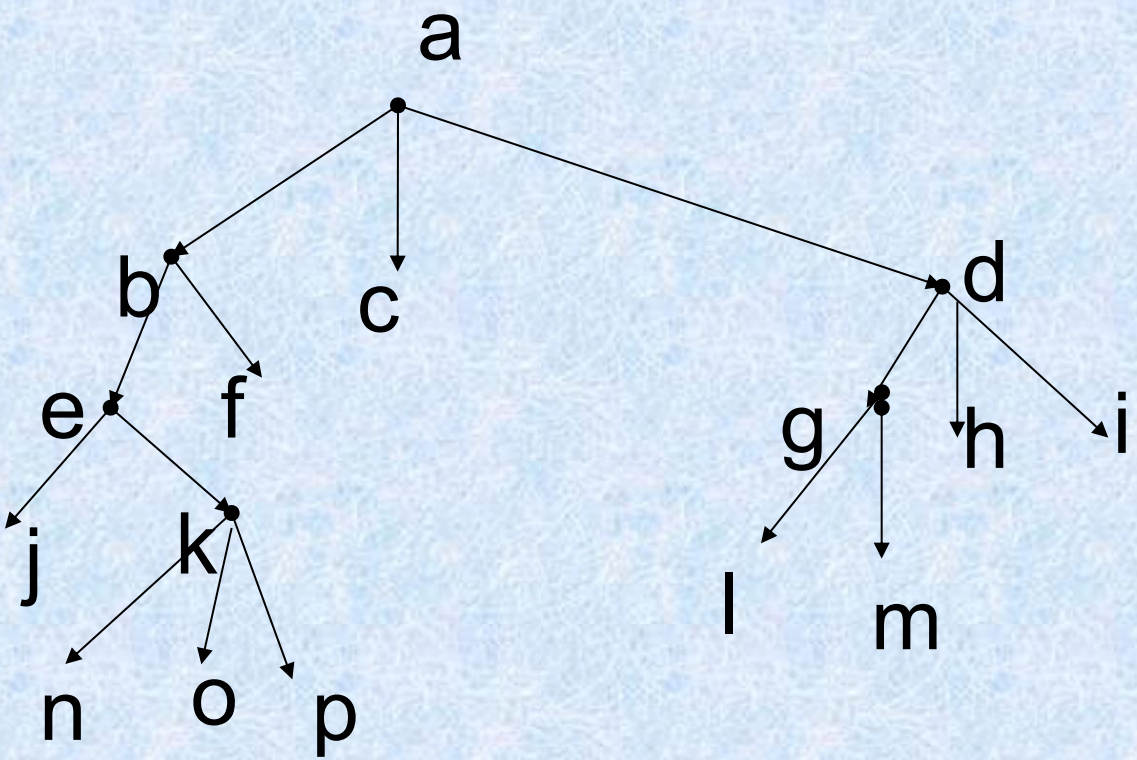


b）中序：j,e,n,k,o,p,b,f,a,c,l,g,m,d,h,i

Traversal Algorithms

- Definition 3:Let *T* be an ordered rooted tree with root *r*. If T consists only of *r*, then *r* is the **postorder traversal** of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are subtrees at *r* from left to right in *T*. The postorder traversal begins by traversing T_1 in postorder. then T_2 in postorder, and so on, then T_n is traversed in postorder,and ends by visiting r.

postorder traversal



c ）后序：j,n,o,p,k,e,f,b,c,l,m,g,h,i ,d,a

二元树的遍历问题:

（或周游/traversal：即访问一遍所有顶点）

有三种访问方法：

1. 前序周游/preorder traversal：

先游根，

再左子树，

后右子树。

2. 中序周游/inorder traversal：

先游左子树，

再根，

后右子树。

3. 后序周游/**postorder** traversal：

先游左子树，

再右子树，

最后游根。

11.3 Tree Traversal

- Visiting:performing appropriate tasks at a vertex.
- Searching or tree search :the process of visiting each vertex of a tree in some specific order.(walking or traversing（遍历）)
- Left subtree（左子树）:$T(v_L)$
- Right subtree （右子树） :$T(v_R)$

Preorder search（前序遍历）

- Algorithm preorder
  - Step 1　visit v.
  - Step 2　if v_L exists,then apply this algorithm to $(T(v_L), v_L)$.
  - Step 3　if v_R exists,then apply this algorithm to $(T(v_R), v_R)$.
  - 　end of algorithm

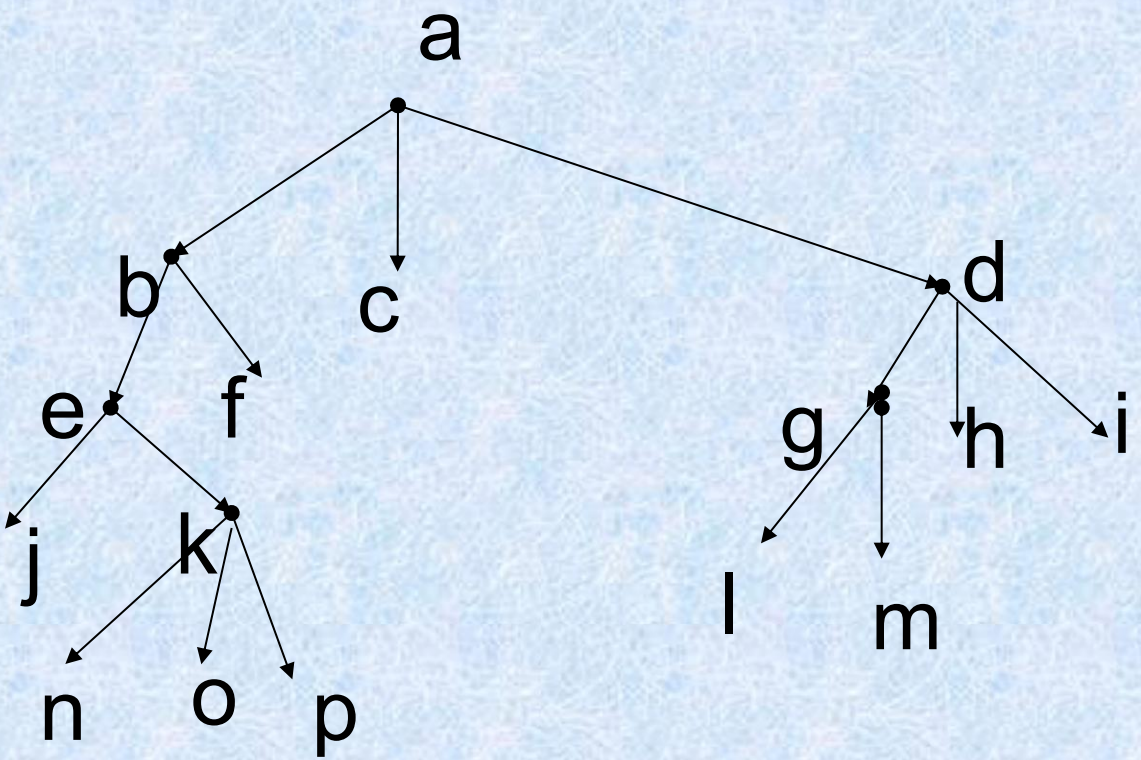
Inorder（中序遍历）

- Algorithm inorder
  - Step 1 search the left subtree($T(v_L)$, v_L), if it exists.
  - Step 2 visit the root v.
  - Step 3 search the right subtree($T(v_R)$, v_R), if it exists.
  - end of algorithm

Postorder（后序遍历）

- Algorithm postorder
  - Step 1 search the left subtree($T(v_L)$, v_L), if it exists.
  - Step 2. search the right subtree($T(v_R)$, v_R), if it exists.
  - Step 3. visit the root v.
  - end of algorithm

11.3 Tree Traversal



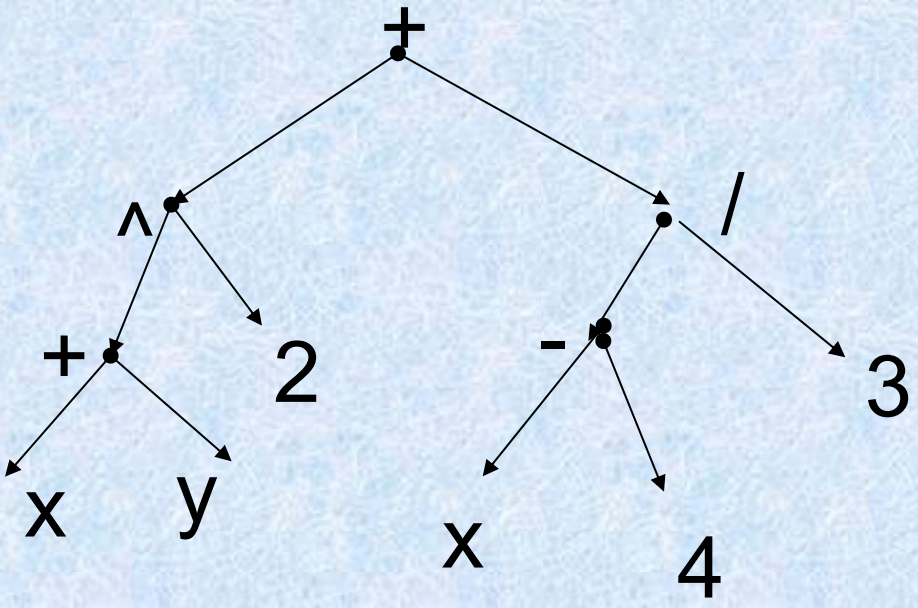
a）前序：a,b,e,j,k,n,o,p,f, c,d,g,l,m,h,i

b）中序：j,e,n,k,o,p,b,f,a,c,l,g,m,d,h,i

c）后序：j,n,o,p,k,e,f,b,c,l,m,g,h,i ,d,a

Infix,Prefix,and Postfix Notation

A Binary Three Representing((x+y)^2+((x-4)/3)



a ）Prefix：+^+xy2/-x43

b ）Infix：x+y^2+x-4/3

c ）Postfix：xy+2^x4-3/+

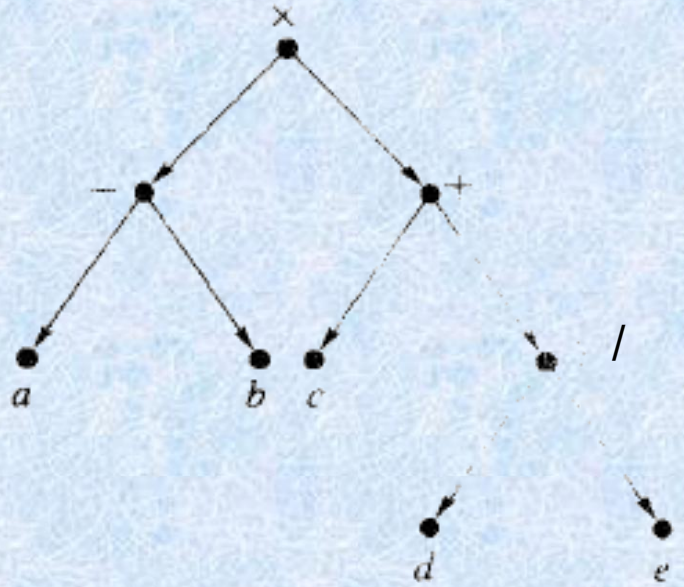
Prefix(Polish Form)
（前缀表达式（波兰式））

- Unambiguously without parentheses
- from left to right　Fxy
- operation symbol precedes the argument

1. $\times - 6\ 4 + 5 \div 2\ 2$
2. $\times\ 2 + 5 \div 2\ 2$　since the first string of the correct type is $-\ 6\ 4$ and $6 - 4 = 2$
3. $\times\ 2 + 5\ 1$　replacing $\div\ 2\ 2$ by $2 \div 2$ or 1
4. $\times\ 2\ 6$　replacing $+\ 5\ 1$ by $5 + 1$ or 6
5. 12　replacing $\times\ 2\ 6$ by 2×6

Infix notation（中缀表示法）

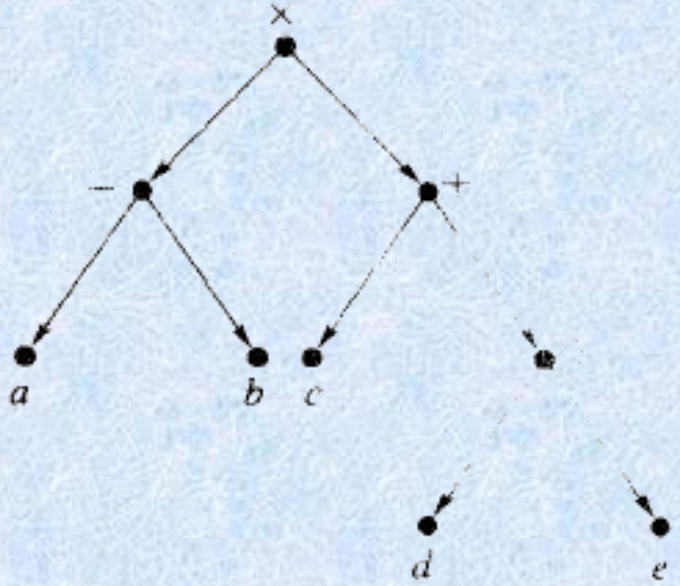
- Ambiguous



(a)

Figure 7.14

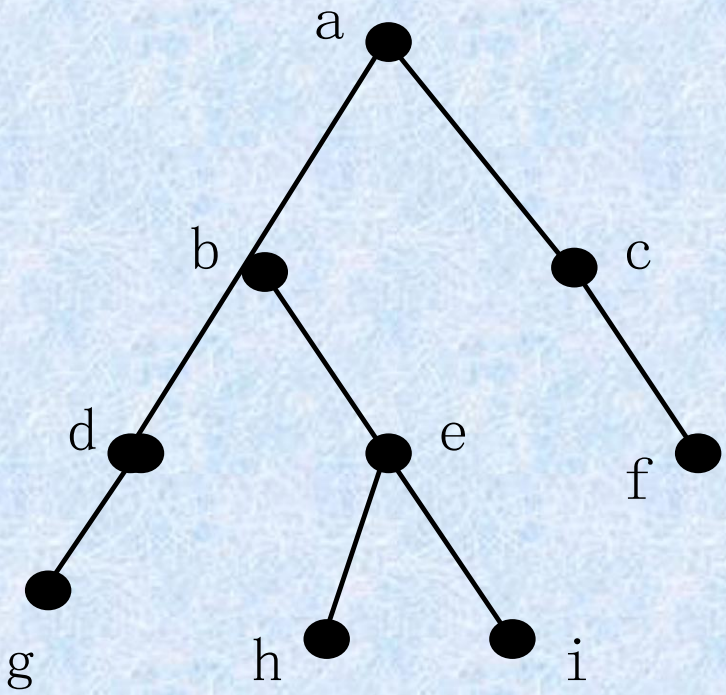
Postfix（后缀表达式）



(a)

Figure 7.14

- ab-cde/+*
- a=2,b=1,c=3



a）前序：a，b，d，g，e，h，i，c，f

b）中序：g，d，b，h，e，i，a，c，f

c）后序：g，d，h，i，e，b，f，c，a

二元树的遍历问题的应用

公式的三种表示方法：

1. Infix notation

2. Prefix notation
   Polish notation

3. **Postfix notation**
   reverse Polish notation

11.4 Spanning Trees （生成树）

- Let G be a simple graph A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.
  - T is a tree with exactly the same vertices as G
  - T can be obtained from G by deleting some edges of G.

Example 3

- Spanning trees are not unique.

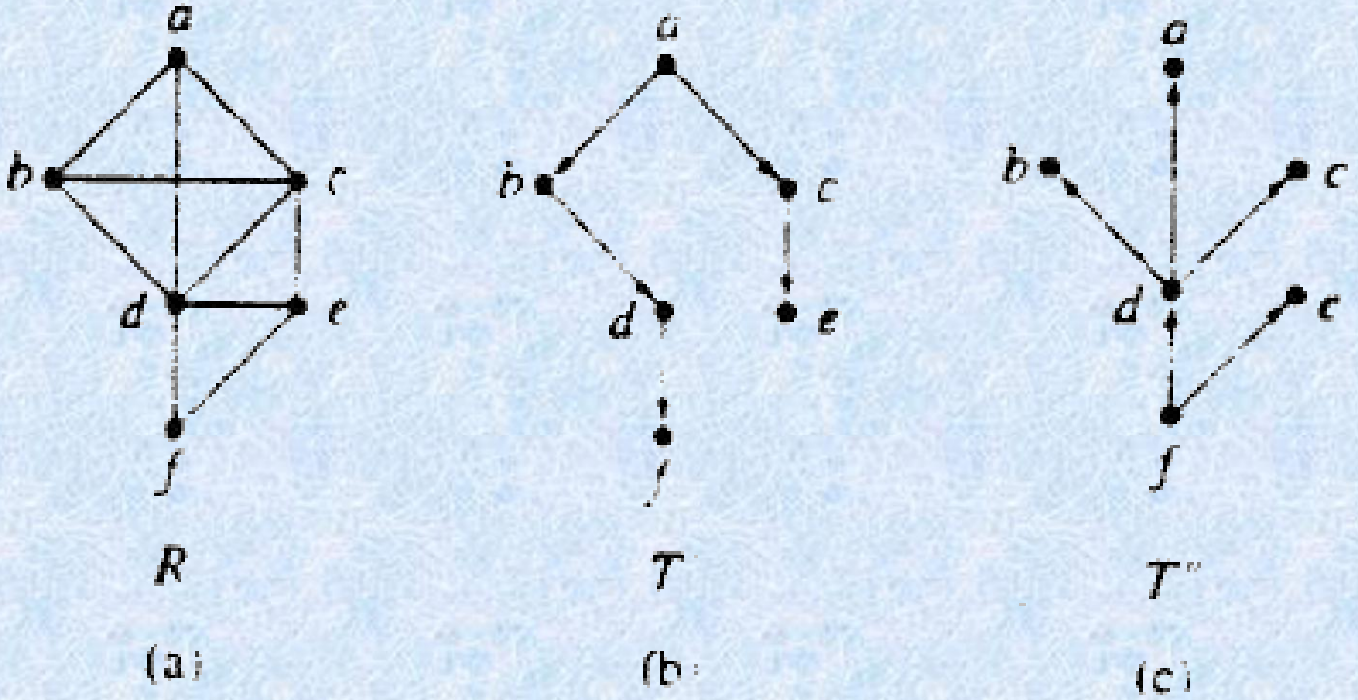


Figure 7.30

定义 G的生成树:G的是一棵树的生成子图.

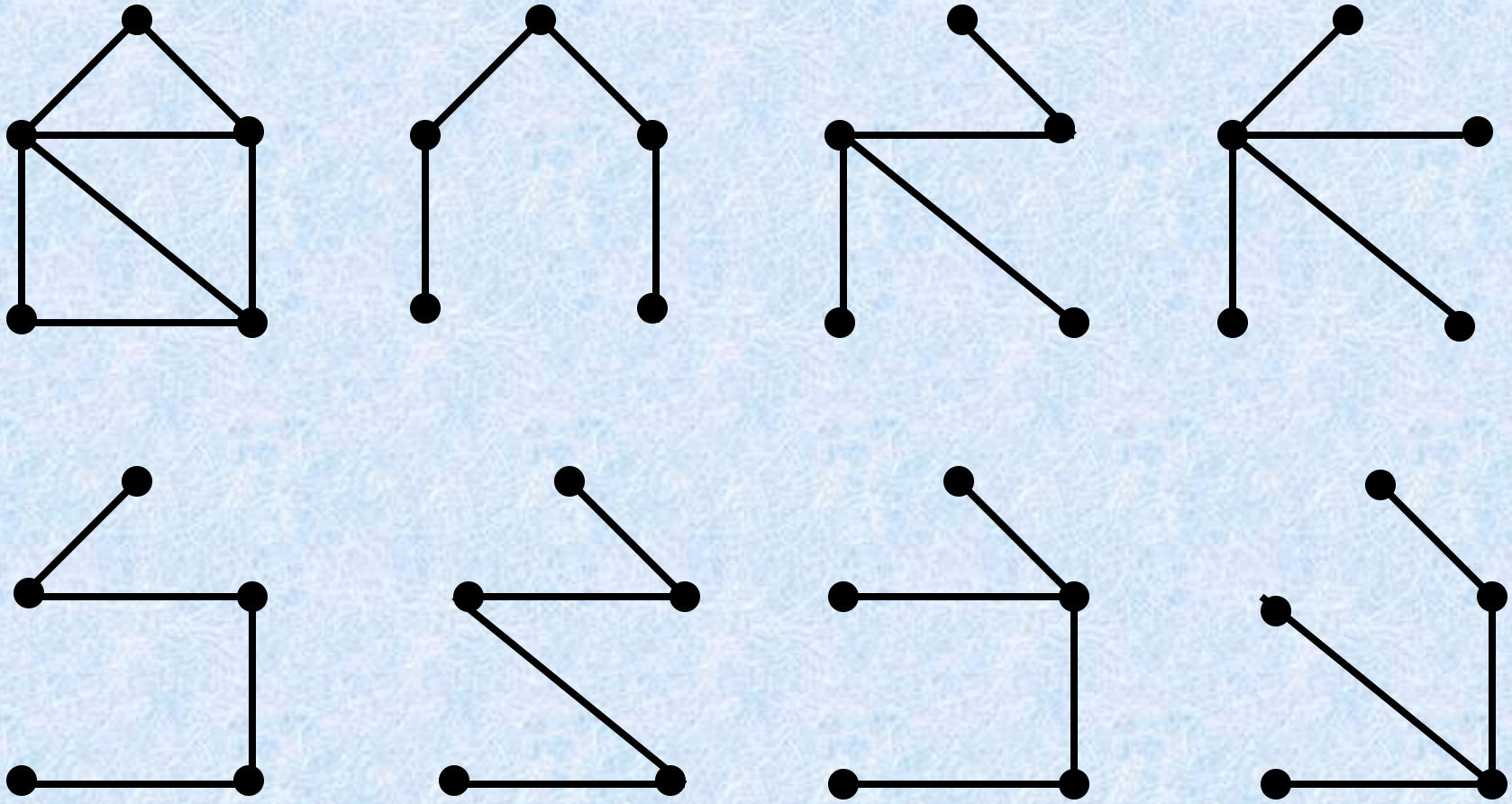
树枝: 生成树T中的边;

弦: G的不在生成树中的边;

生成树T的补: 所有弦的集合.

定理 连通图至少有一棵生成树.

一个连通图可有多棵生成树.



[定义]生成树：无向简单图G＝（V，E）的生成子图T是树，则称T为G的生成树/spanning tree。

T＝（V，E’），　E’⊆E

[定理一]：

无向简单图G＝（V，E）存在生成树的充要条件是G是连通的。

证明：　必要性：T是生成子图，包含G的所有顶点，T是树，T连通，则G连通。

充分性：若G是连通的，

（1）若G无回路，则G本身是树，即生成树。

（2）若存在回路，去掉回路中任一边，不影响连通性，也不减少顶点.

所有回路都移去一条边，剩下的子图包含所有顶点，又是无回路的连通图，即为生成树。

注：① 移去边时可随意，故生成树不唯一。

② 生成树是G的最小连通生成子图，若将G＝（V，E）看成是各顶点间的通讯联络图，则生成树T＝（V，E’）即为各顶点间直接或间接联络的最小通讯联络图。

IP Multicasting

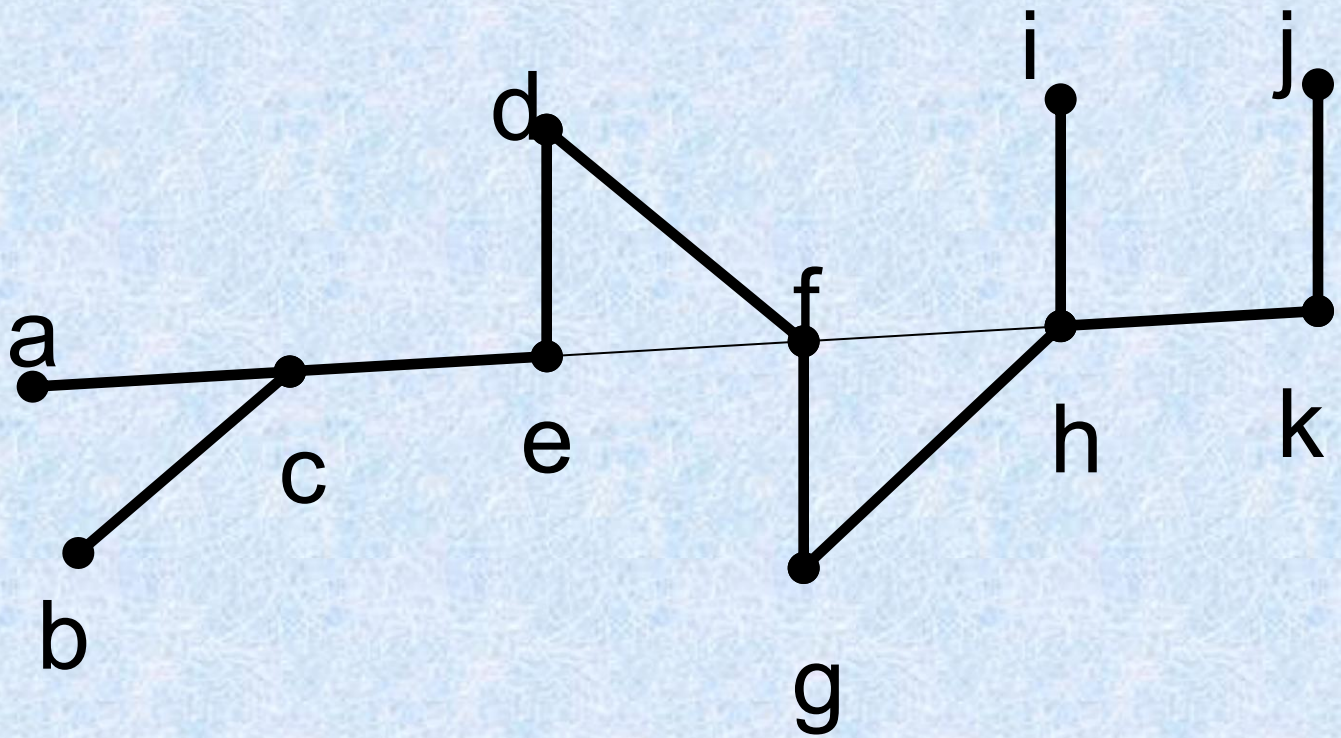
- 目前，使用得最为广泛的组播技术是IP Multicast。IP组播技术是一种为优化使用网络资源而产生的技术，通常用于多点工作方式下的应用程序中，它是标准IP网络层协议技术的一个扩展。

  从Steve Deering于1989年提出的IETF的RFC1112"Host Extension for IP Multicast"中的定义我们可以得知：IP组播的核心思想是--通过一个IP地址向一组主机发送数据（UDP包）。发送者仅仅向一个组地址发送信息，接收者只需加入到这个分组就可以接收信息，所有的接收者接收的是同一个数据流，组中成员是动态的，可以根据自己的意愿随时随意加入或退出。每一台主机都可以同时加入到多个组中，每一个组播地址可以在不同的端口或者不同的套接字（Socket）上有多个数据流，同时许多实际应用可以共享一个组地址。IP组播技术可以有效地避免重复发送可能引起的广播风暴，并且能够突破路由器的限制，将数据包传送到其它网段。

Depth-first search深度优先

- Build a spanning tree for a connected simple graph using a depth-first search.

Depth-first search深度优先

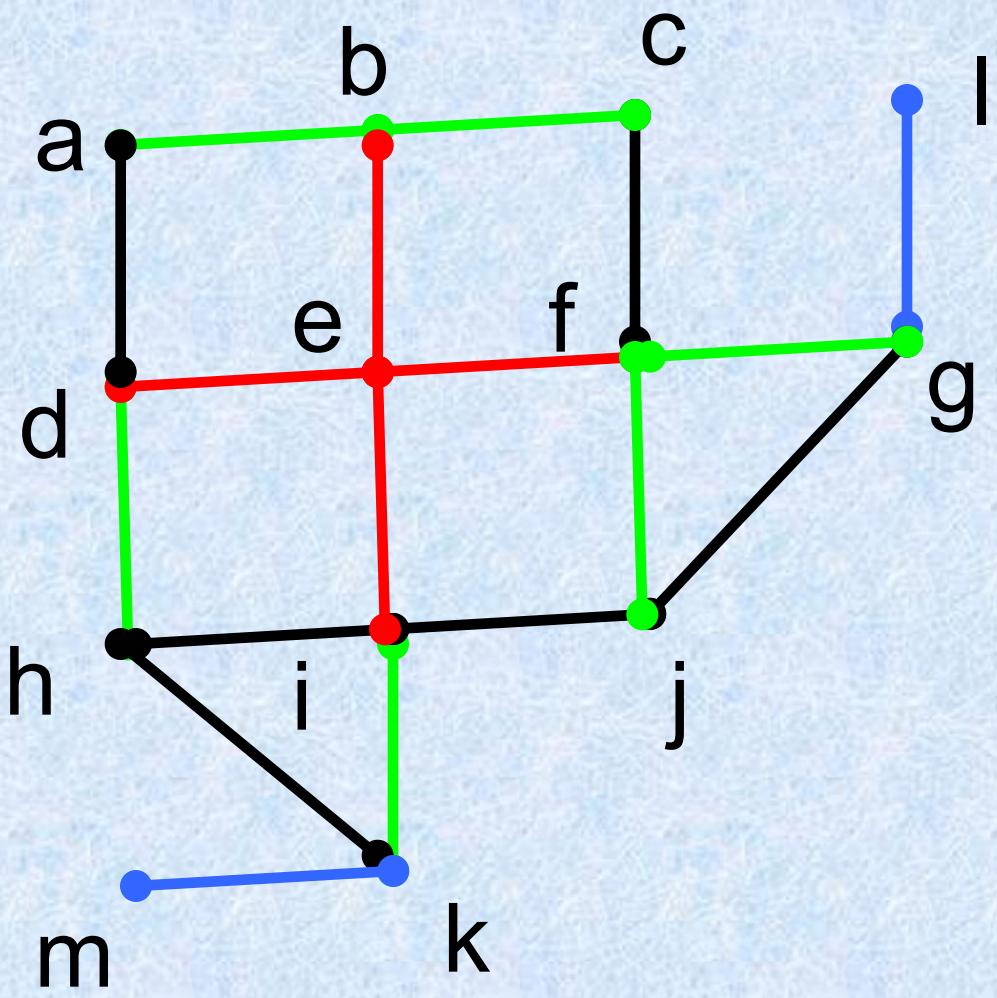


Tree edges back edges

Breadth-first search广度优先

- Produce a spanning tree of a simple graph by the use of a breadth-first search.

Breadth-first search广度优先



Backtracking 回溯法

- Graph Colorings
- The n-Queens Problem

- In directed graphs

How can backtracking be used to decide whether a graph can be colored using *n* colors?

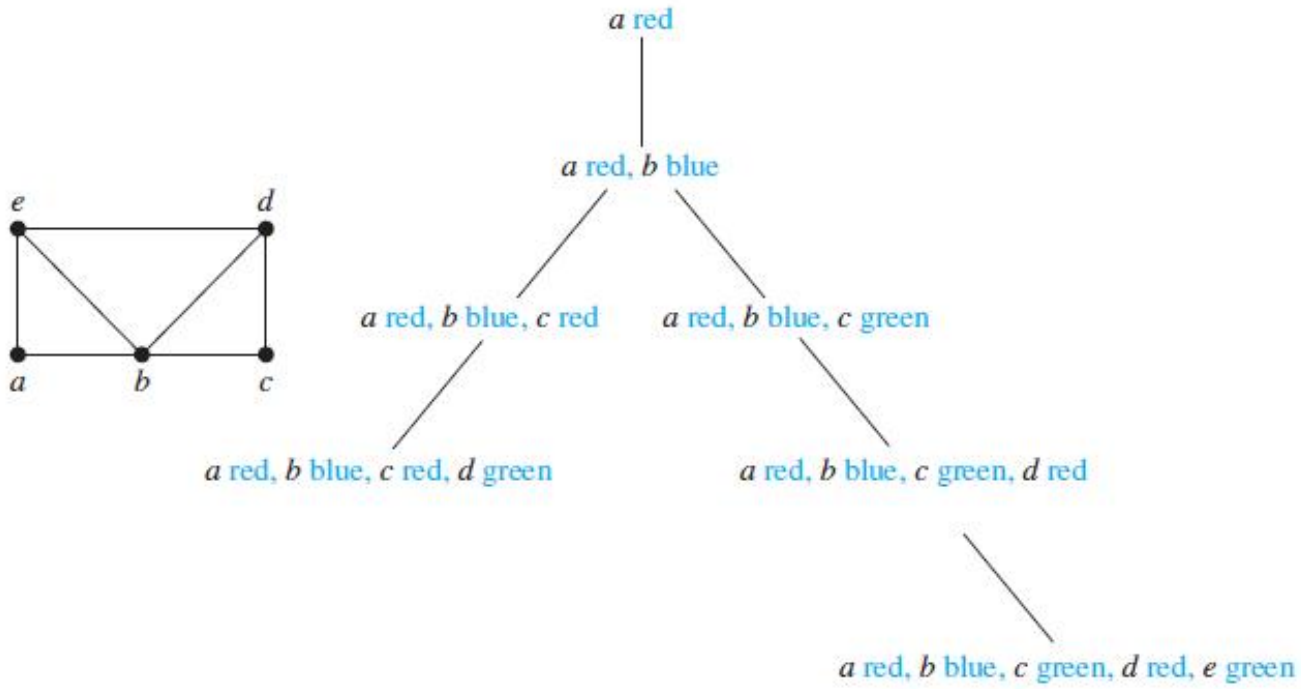


FIGURE 11 **Coloring a graph using backtracking.**

How can backtracking be used to solve the *n*-queens problem?

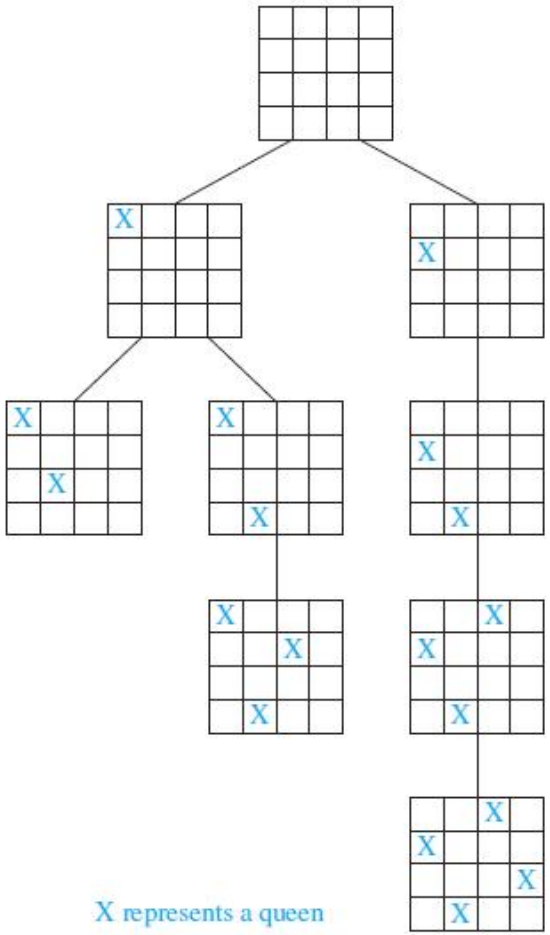


FIGURE 12 **A backtracking solution of the four-queens problem.**

Sums of Subsets Consider this problem. Given a set of positive integers $x_1, x_2, \ldots, x_n$, find a subset of this set of integers that has M as its sum. How can backtracking be used to solve this problem?

Sums of Subsets

a backtracking solution to the problem of finding a subset of {31, 27, 15, 11, 7, 5} with the sum equal to 39.

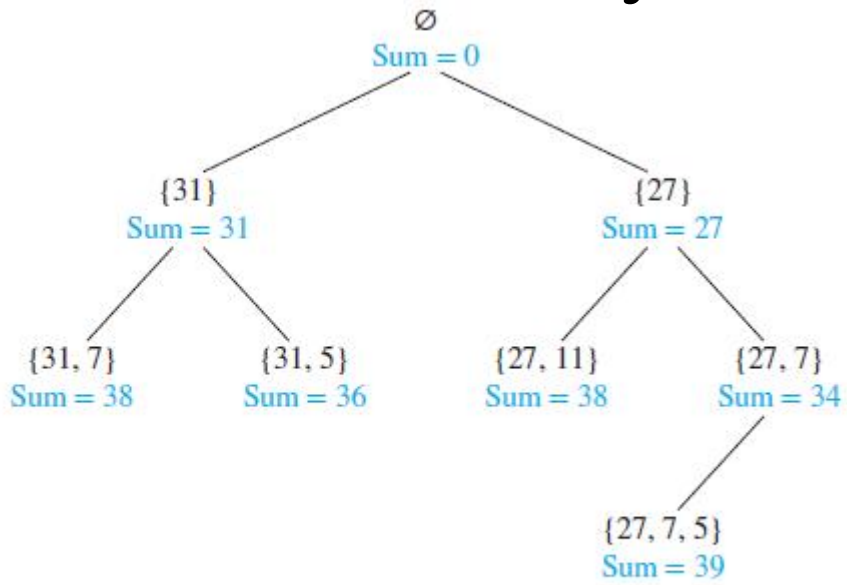


FIGURE 13 **Find a sum equal to 39 using backtracking.**

Web spiders

- **What is a Spider or Search Engine Spider?**
-
- Description: A **Spider** or *Search Engine Spider* is a program that automatically traverses the Web and requests documents from URLs. A spider usually starts from a historical list of URLs and retrieves referenced documents. As it visits new Internet websites it checks to see if the site is already listed in its database. If the site is already listed it usually updates any changes it finds. Spiders are also commonly known as *Robots or Crawlers.* Other names sometimes used: *Webwalkers, Wanderers, or *Worms*

11.5 Minimal Spanning Trees （最小生成树）

Weighted graph（加权图）

- Definition:a **weighted graph** is a graph for which each edge is labeled with a numerical value called its **weight**（权值）.
- Example 1

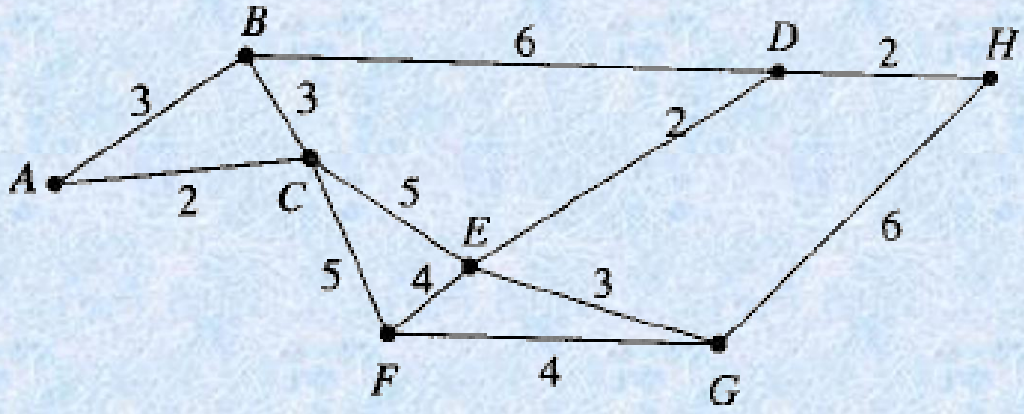


Figure 7.49

Example 2

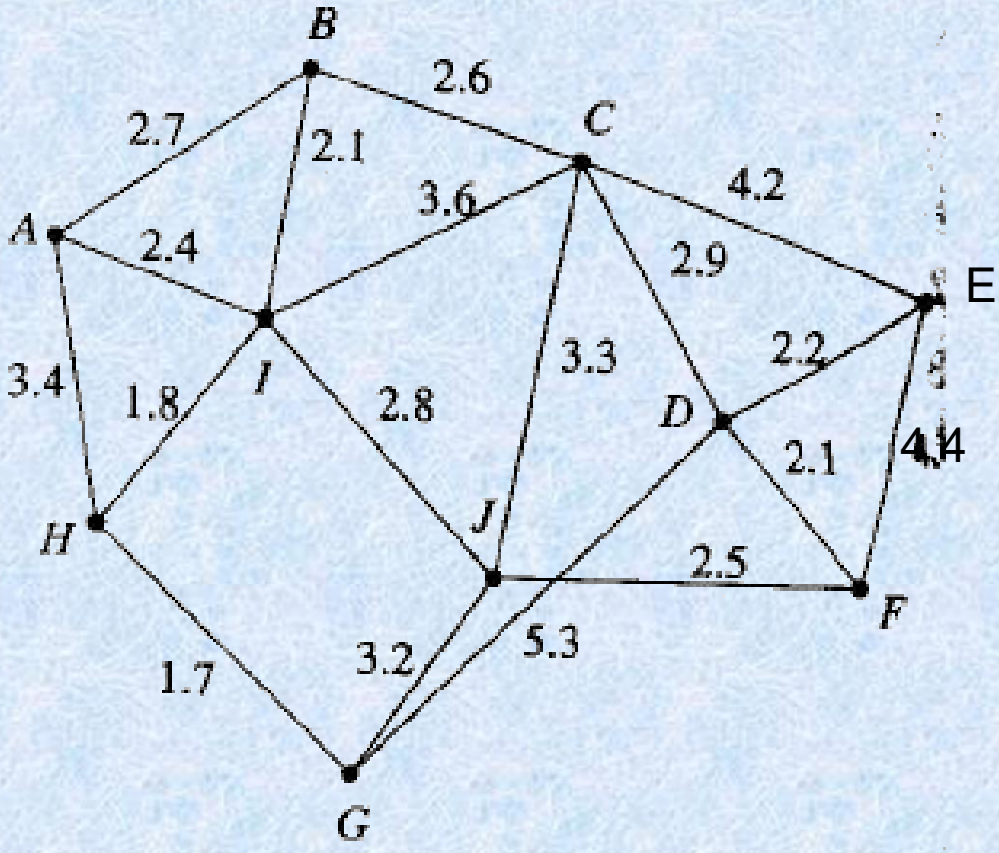


Figure 7.50

- The weight of an edge (v_i,v_j) is some times referred to as the *distance between vertice* v_i and v_j.
- A vertex *u* is a nearest neighbor of vertex *v* if *u* and *v* are adjacent and no other vertex is joined to *v* by an edge of lesser weight than(*u*,*v*).
- Note *v* may have more than one nearest neighbor.

Example 3

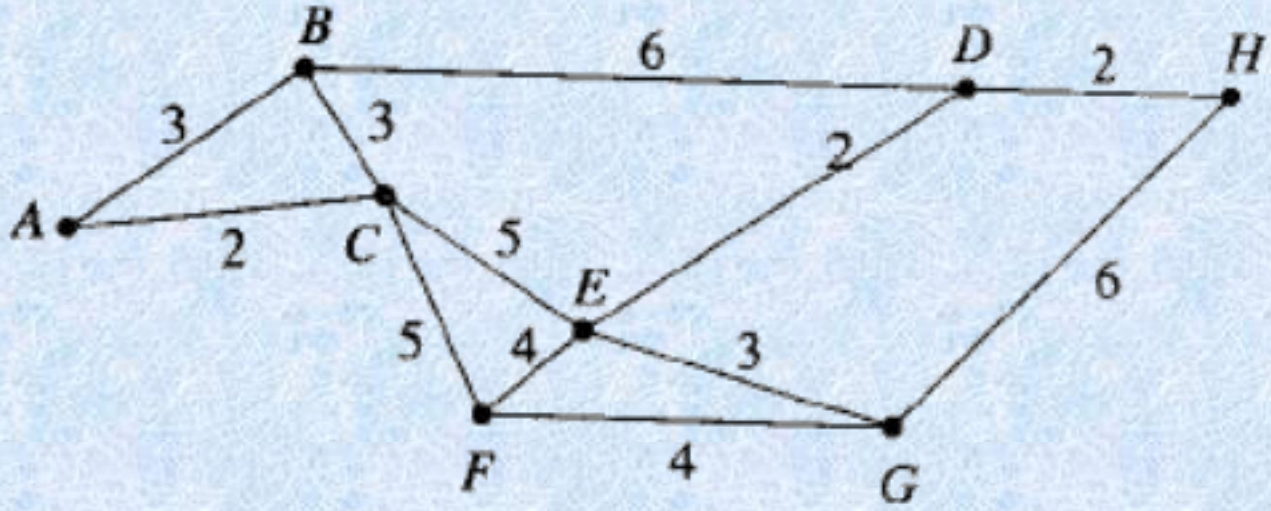


Figure 7.49

- A vertex v is a **nearest neighbor of a set of vertices** $V=\{v_1,v_2,\ldots v_k\}$ in a graph if v is adjacent to some member v_i of V and no other vertex adjacent to a member of V is joined by an edge of lesser weight than (v,v_i).
- This vertex v may belong to V.

Example 4

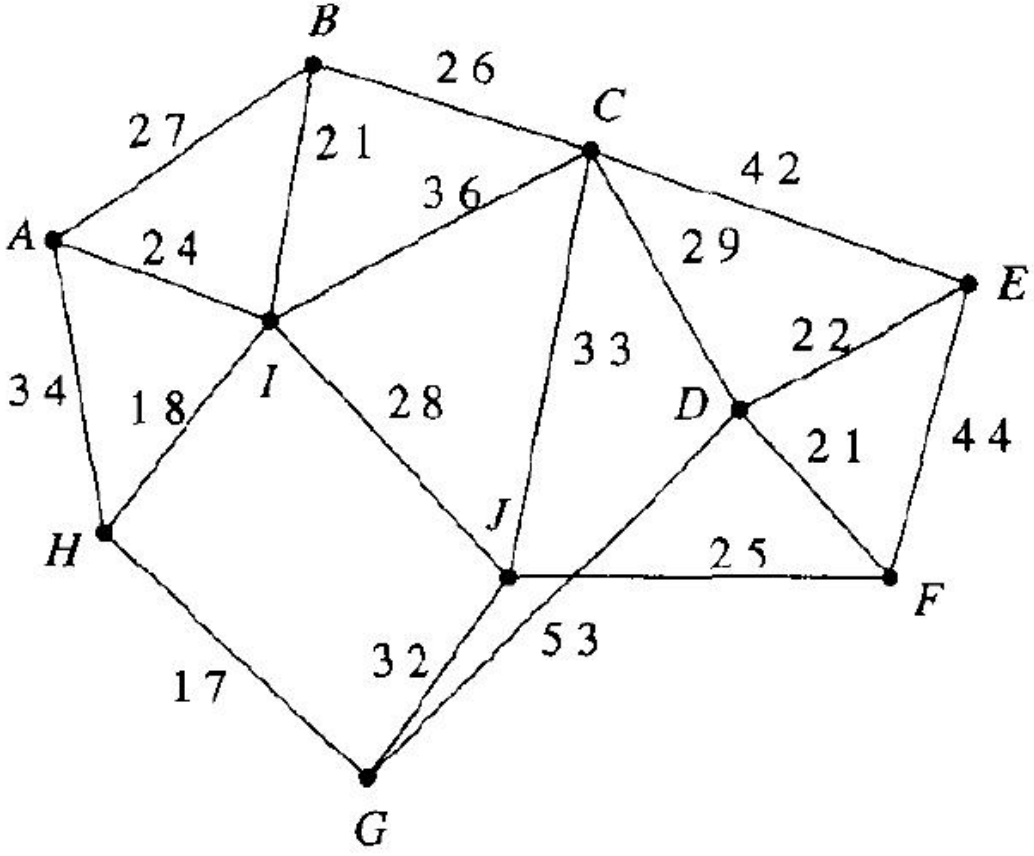


Figure 7 50

V={C,E,J}

D is the nearest neighbor of V.

Minimal spanning tree
（最小生成树）

- Definition: an undirected spanning tree of a weighted graphs,for which the total weight of the edges in the tree is as small as possible. Such a spanning tree is called a minimal spanning tree.

Prim's algorithm

- **Procedure** Prim(*G*:weighted connected undirected graph with n vertices)
- *T*:= a minimum-weighted edge
- For *i*:=*1* to *n-2*
- **Begin**
- *e*:=an edge of minimum weight incident to a vertex in *T* and not forming a simple circuit in *T* if added to *T*
- T:= T with e added
- **End**{*T* is a minimum spanning tree of *G*}

Example 5

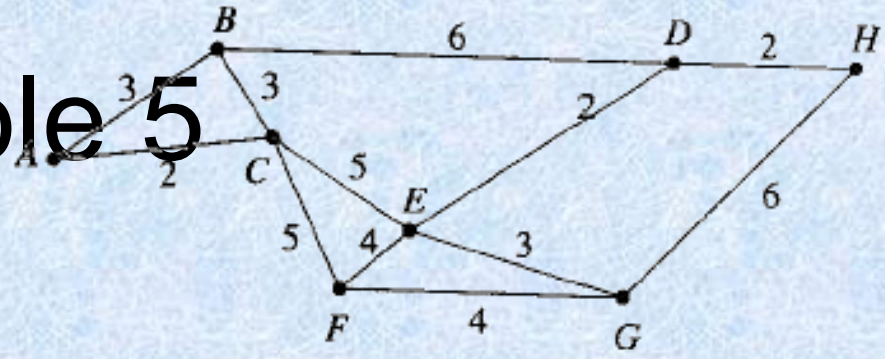
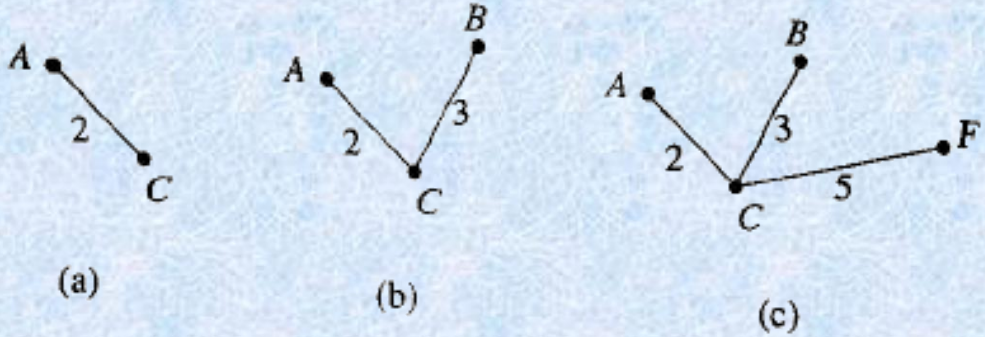


Figure 7.49



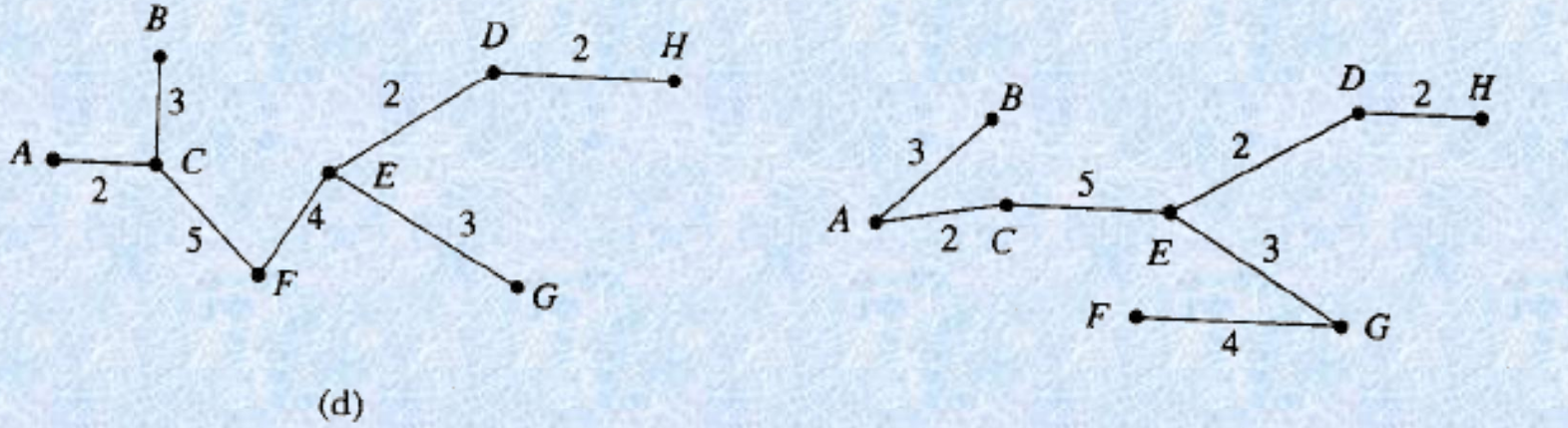


Figure 7.52

Example 6

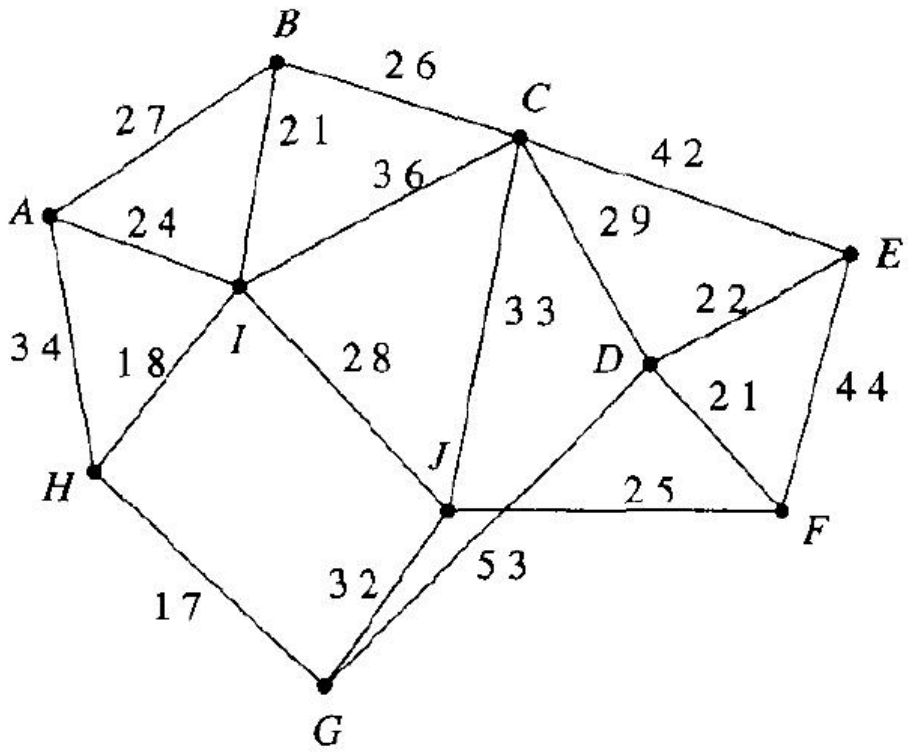


Figure 7.50

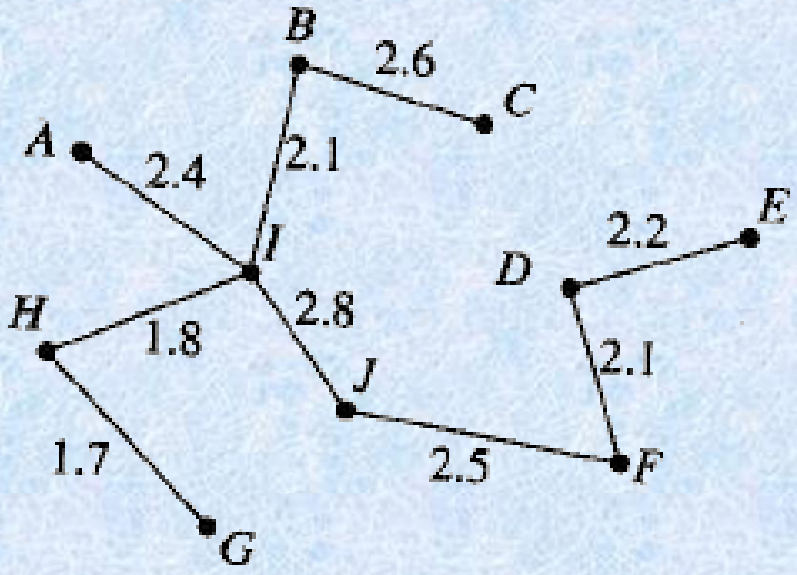
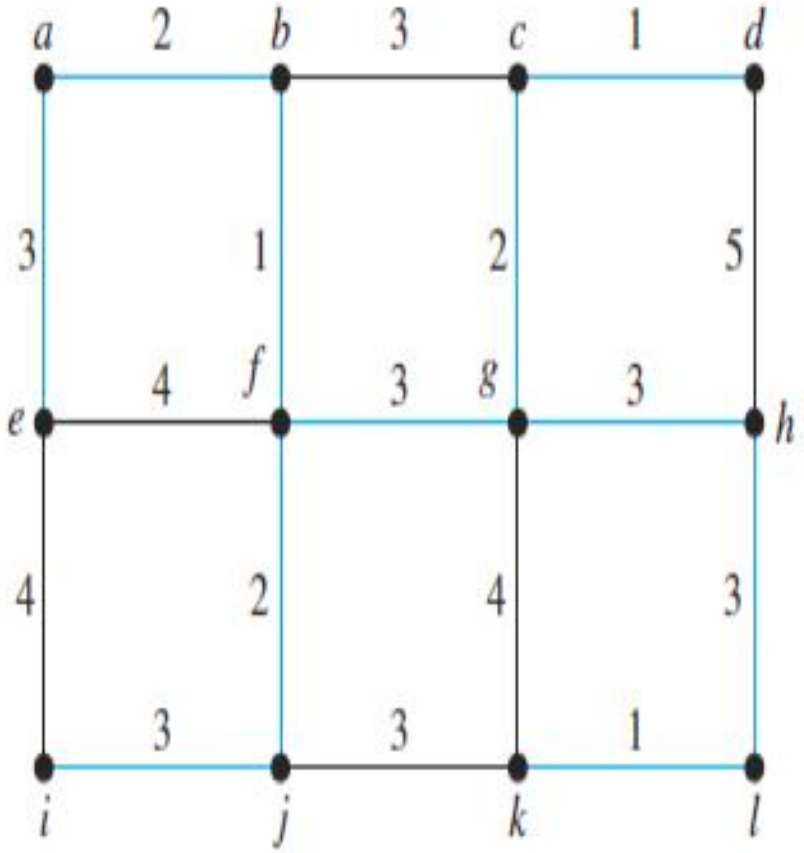


Figure 7.53



(a)

Choice	Edge	Weight
1	$\{b, f\}$	1
2	$\{a, b\}$	2
3	$\{f, j\}$	2
4	$\{a, e\}$	3
5	$\{i, j\}$	3
6	$\{f, g\}$	3
7	$\{c, g\}$	2
8	$\{c, d\}$	1
9	$\{g, h\}$	3
10	$\{h, l\}$	3
11	$\{k, l\}$	1
	Total:	24

(b)

FIGURE 4 **A Minimum Spanning Tree Produced Using Prim's Algorithm.**

Theorem 1

- Prim's algorithm produces a minimal spanning tree for the graph G.
- Proof
  - Let G have n vertex. Let T be the spanning tree for G produced by Prim's algorithm.
  - Suppose that the edges of T are $t_1, t_2, \ldots, t_{n-1}$ in the order of they were selected.
  - we define T_i (i=1 to n-1) to be the tree with $t_1, t_2, \ldots, t_i$ and $T_0=\{\}$Then $T_0 \subset T_1 \ldots \subset T_{n-1}=T$.
  - We now proved that each T_i is contained in a minimal spanning tree for G by mathematical induction.

- Basis step: Clearly P(0):T_0={} is contained in every minimal spanning tree for G
- Induction Step:
  - suppose P(k):T_k is contained in a minimal spanning tree T’ for G.
  - By definition $\{t_1, t_2,\ldots,t_k\} \subseteq T'$. If t_{k+1}also belongs to T’,then $T_{k+} \subseteq T'$ and we have P(k+1) is true.
  - If t_{k+1}does not belongs to T’,then $T' \cup \{t_{k+1}\}$must contain a cycle.
  - This cycle would be for some edges $s_1, s_2,\ldots,s_k$ in T’
  - The edges of this cycle cannot all be fromT_k or T_{k+1} would contain this cycle.

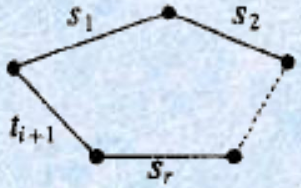
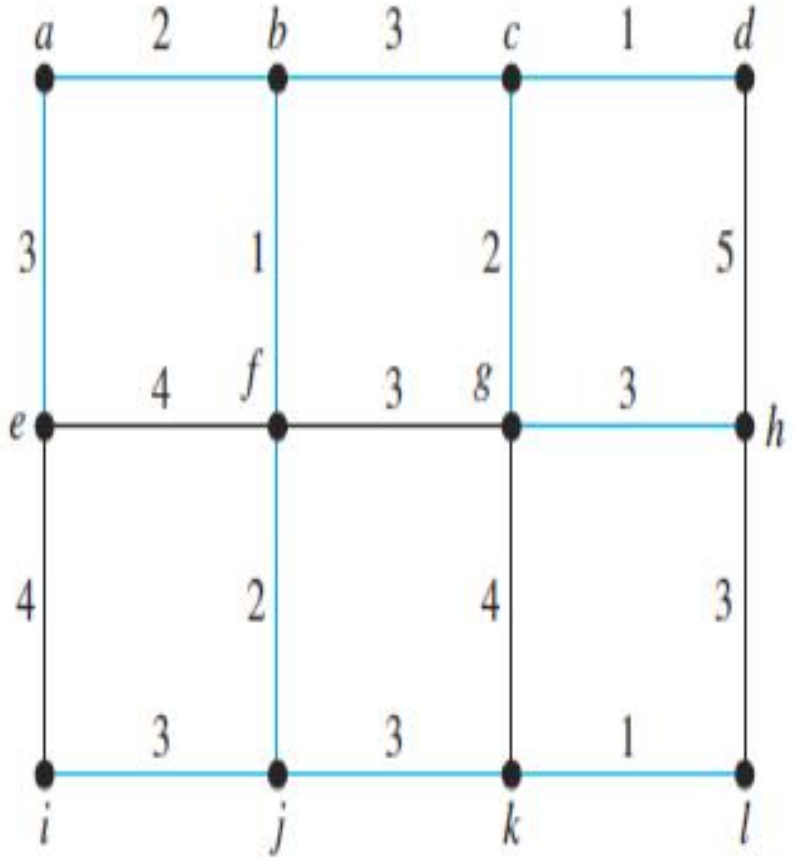


Figure 7.51

- Let s_l be the edge with smallest index l that is not in T_k.when t_{k+1} was chosen by Prim‘s algorithm, s_l was also available.thus the weight of s_l is at least as large as that of t_{k+1}.
- The spanning tree (T’ -{s_l}) $\cup${t_{k+1} }contains T_{k+1}. The weight of this tree is less than or equal to the weight of T’ ,so it is a minimal tree for G.
- P(k+1) is true. So T_{n-1}=T is contained in a minimal spanning tree and must in fact be that minimal spanning tree.
- Q.E.D

Kruskal's algorithm

- Let G be a weighted connected undirected graph with n vertices and let $S=\{e_1, e_2,\ldots, e_k\}$ be the set of weighted edges of G.
  - Step 1 Choose an edge e_1 in S of least weight.Let $E=\{e_1\}$ Replace S with $S-\{e_1\}$.
  - Step 2 Choose an edge e_i in S of least weight that will not make a cycle with members of E.Replace E with $E \cup \{e_i\}$ and S with $S-\{e_i\}$.
  - Step 3 Repeat step 2 until $|E|=n-1$.
  - End of algorithm



(a)

Choice	Edge	Weight
1	{c, d}	1
2	{k, l}	1
3	{b, f}	1
4	{c, g}	2
5	{a, b}	2
6	{f, j}	2
7	{b, c}	3
8	{j, k}	3
9	{g, h}	3
10	{i, j}	3
11	{a, e}	3
	Total:	24

(b)

FIGURE 5 A Minimum Spanning Tree Produced by Kruskal's Algorithm.

Example 7

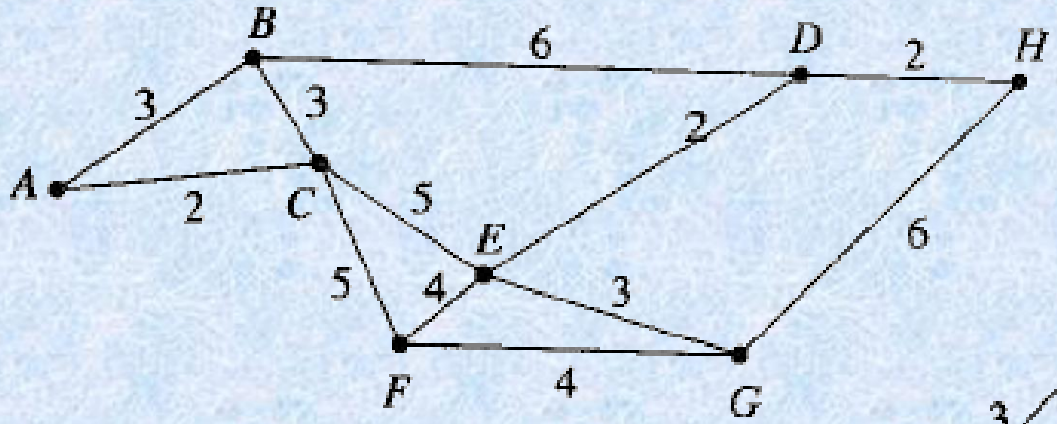


Figure 7.49

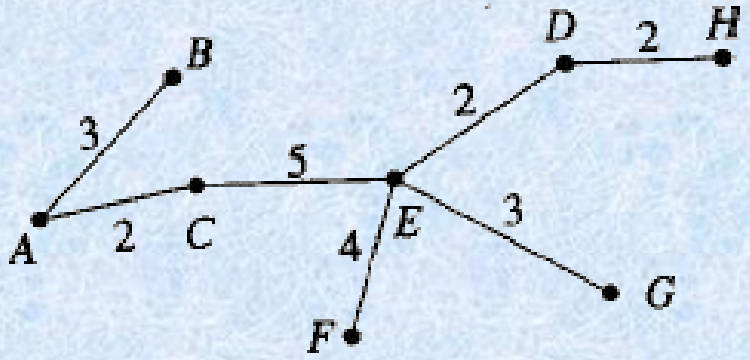


Figure 7.54

Example 8

- Use Kruskal's algorithm to find a minimal spanning tree for the graph

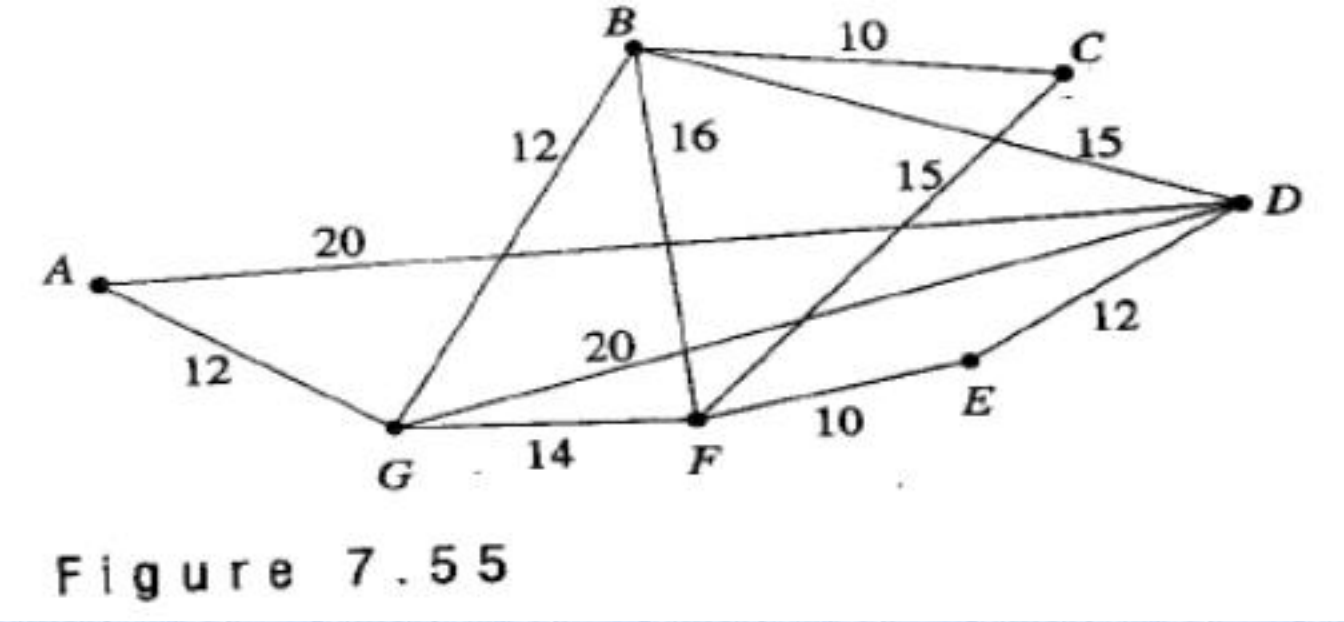
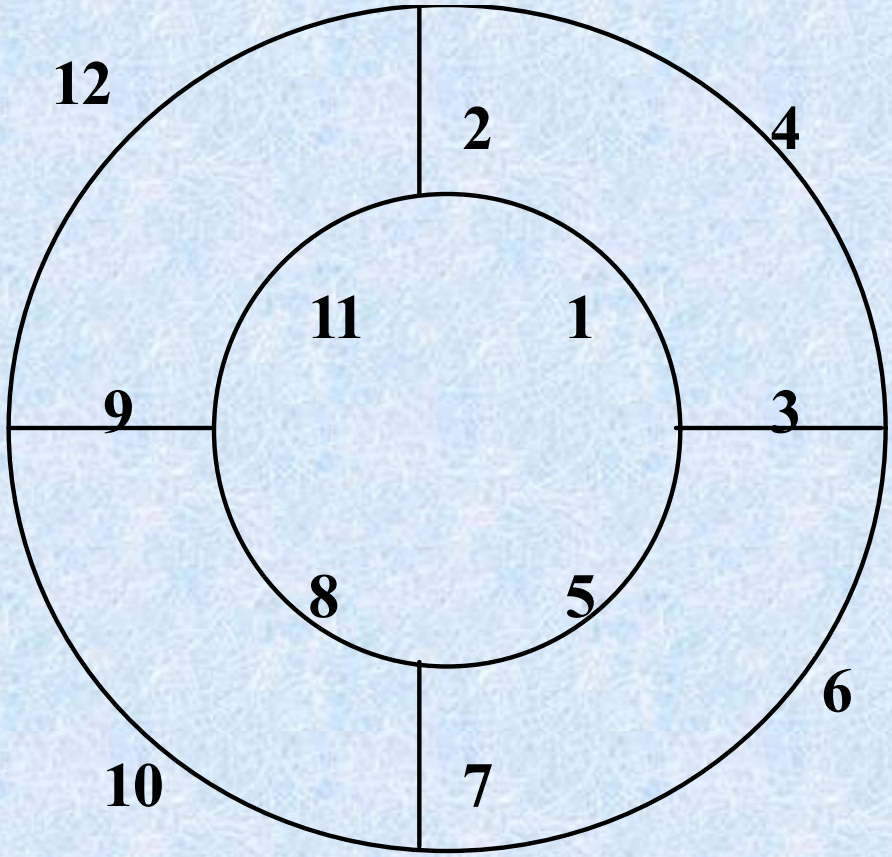


Figure 7.55


12
2
4
11
1
9
3
8
5
6
10
7

解题过程：

(1）边排序:(v1,v2),(v2,v7),(v1,v6),(v6,v7)

(v1,v4),(v5,v6),(v4,v5),(v3,v4),(v3,v8)

(v5,v8),(v2,v3),(v7,v8)

(2) 选s1={e1} e1=(v1,v2)

(3) 选e2=(v2,v7)

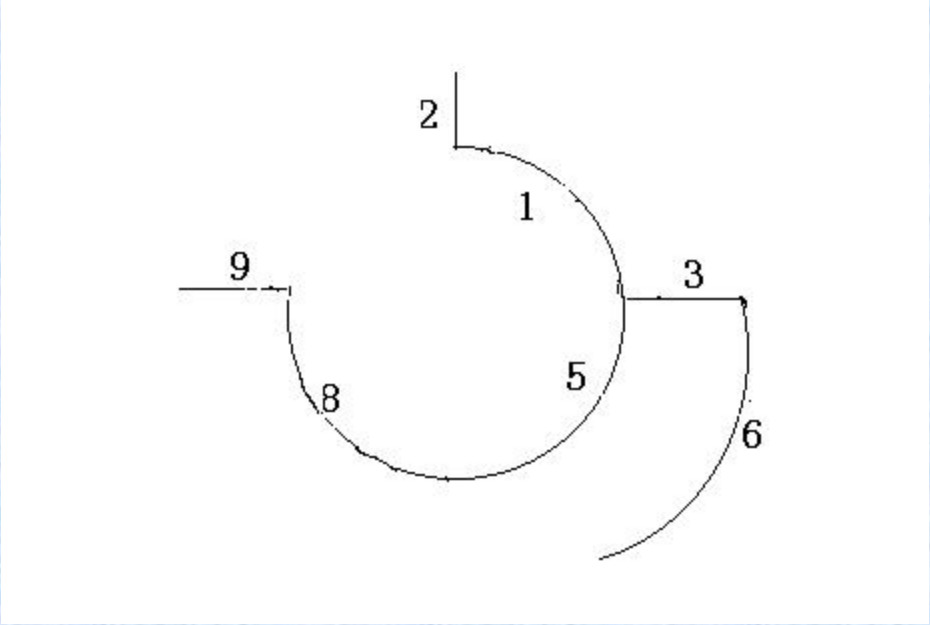
(4) e3=(v1,v6)

(5) e4=(v1,v4)=5

(6) e5=(v5,v6)=6

(7) e6=(v3,v4)=8

(8) e7=(v3,v8)

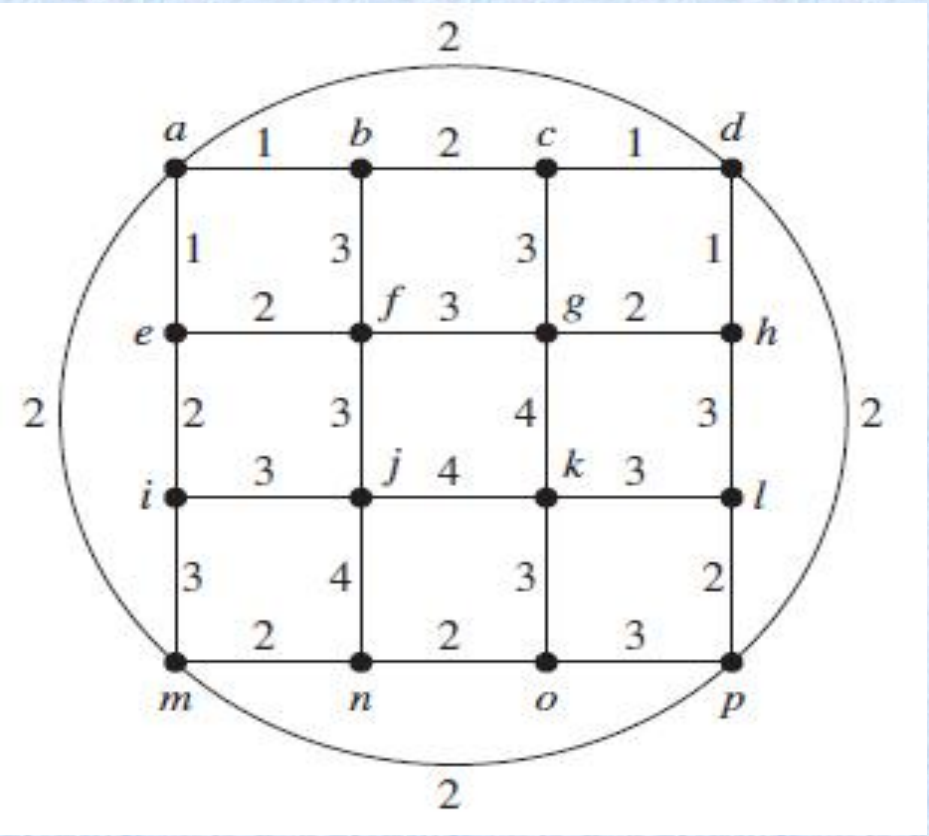


作业

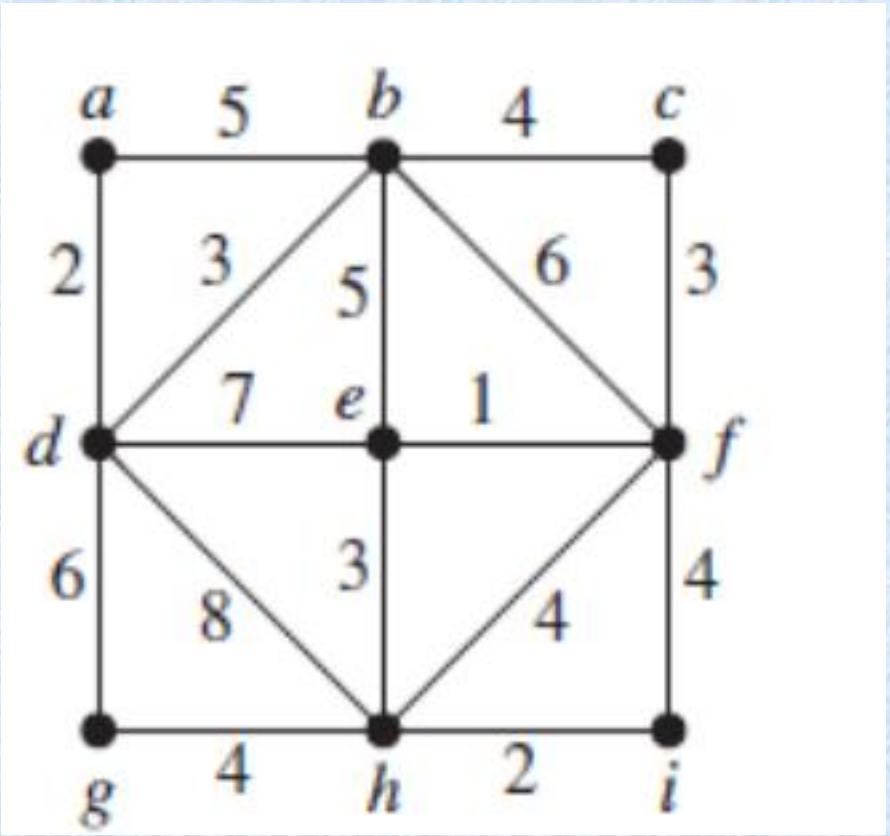
- § 11.3 – 6, 18, 22, 34
- § 11.4 – 12, 14, 22, 24, 50, 54
- § 11.5 – 4, 8, 10, 14, 18

- Distance between two spanning trees

Use Prim's algorithm to find a minimum spanning tree for the given weighted graph G1.
Use Kruskal's algorithm to find a minimum spanning tree for the weighted graph G2.



G1



G2